

长广溪智能制造（无锡）有限公司

Changuangxi Intelligent Manufacturing (Wuxi) Co., Ltd.

协作机器人脚本手册

文档版本：V1.4

发布日期：2025 年 10 月 27 日

文档编号：CGXi-QM-CR-009



长广溪智能制造（无锡）有限公司

Changuangxi Intelligent Manufacturing(Wuxi) Co., Ltd.



长广溪智能制造（无锡）有限公司（简称长广溪智造）保留对此文件修改之权利且不另行通知。长广溪智能制造（无锡）有限公司所提供之信息相信为正确且可靠之信息，但并不保证本文件中绝无错误。请于向长广溪智能制造（无锡）有限公司提出订单前，自行确定所使用之相关技术文件及规格为最新之版本。若因贵公司使用本公司之文件或产品，而涉及第三人之专利或著作权等智能财产权之应用及配合时，则应由贵公司负责取得同意及授权，本公司仅单纯販售产品，上述关于同意及授权，非属本公司应为保证之责任。又未经长广溪智能制造（无锡）有限公司之正式书面许可，本公司之所有产品不得使用于医疗器材，维持生命系统及飞航等相关设备。

Copyright © 2022-2023 长广溪智造保留所有版权

前言

本手册将介绍 CGXi 机器人脚本代码的功能及使用方法。在实际编写前，应仔细阅读此手册，在充分理解的基础上方可编写脚本程序。

实际编写程序时，有关于机器人具体功能的程序应用，请浏览《CGXi-协作机器人用户手册（软件篇）》、《CGXi 通讯手册》。

另外，请妥善保管本手册，以便随时能够查阅。

所有手册文件均可登陆长广溪机器人官方网站下载 <https://www.cgxi.com/>。

目录

前言	1
目录	2
一、机器人脚本语言概述	13
1.1 脚本语言介绍	13
1.2 Lua 简介	13
1.3 编程环境及配置要求	13
二、脚本语言编程基础	14
2.1 数据类型	14
2.1.1 number 数字	14
2.1.2 string 类型	14
2.1.3 table (表)	15
2.2 算术运算	16
2.3 位操作	16
2.4 注释	17
2.5 判断及关系运算符	17
2.6 循环	19
2.6.1 for 循环	20
2.6.2 while 循环	21
2.6.3 repeat 循环	21
2.6.4 break 语句	22
2.6.5 goto 和 label	22
2.7 函数	22
2.7.1 函数定义	23
2.7.2 多返回值	23
2.7.3 可变参数	24
2.8 常用函数	25
2.8.1 print	25

三、函数分类	27
3.1 运动函数	27
3.1.1 基础运动函数	27
3.1.1.1 movej	27
3.1.1.2 movejp	27
3.1.1.3 movel	28
3.1.1.4 movec	29
3.1.1.5 addwaypoint	29
3.1.1.6 movex	30
3.1.1.7 moveps	30
3.1.1.8 moveps_end	31
3.1.1.9 movestop	32
3.1.1.10 set_freedrive	32
3.1.1.11 get_freedrive	33
3.1.1.12 set_jerk_acc_ratio	33
3.1.2 传送带功能函数	33
3.1.2.1 conv_deact	33
3.1.2.2 conv_act	33
3.1.2.3 start_conv_track	34
3.1.2.4 stop_conv_track	35
3.1.2.5 movel_wobj	35
3.1.2.6 movec_wobj	36
3.1.3 力控碰撞功能函数	37
3.1.3.1 start_force_mode	37
3.1.3.2 end_force_mode	38
3.1.3.3 force_mode_payload	38
3.1.3.4 force_mode_mass	38
3.1.3.5 force_mode_damping	39

3.1.3.6 force_mode_stiffness	39
3.1.3.7 start_freedrive_mode	40
3.1.3.8 end_freedrive_mode	40
3.1.3.9 get_force_mode_wrench	41
3.1.3.10 is_force_mode	41
3.1.3.11 force_mode_tcp	41
3.1.3.12 set_collision_level	42
3.1.3.13 set_limits_type	42
3.1.4 轨迹再现功能函数	42
3.1.4.1 moves	42
3.1.4.2 check_index	43
3.1.4.3 start_record	44
3.1.4.4 stop_record	44
3.1.4.5 path_save	45
3.1.4.6 get_num_of_wp	45
3.1.4.7 get_first_wp	45
3.1.4.8 get_path_play_index	46
3.1.4.9 get_num_of_path	46
3.1.5 外部轴功能函数	46
3.1.5.1 exj_config_ratio	46
3.1.5.2 exj_config_range	47
3.1.5.3 exj_config_zero	47
3.1.5.4 exj_config_type	47
3.1.5.5 exj_config_coordsys	48
3.1.5.6 exj_config_master	48
3.1.5.7 exj_enable	48
3.1.5.8 exj_disable	49
3.1.5.9 exj_set_pos	49

3.1.5.10	exj_get_pos	49
3.1.5.11	exj_get_servo_pos	50
3.1.5.12	exj_get_servo_pos64	50
3.1.5.13	exj_move	50
3.1.5.14	exj_stop	51
3.1.5.15	exj_continue	51
3.1.5.16	move1_exj	51
3.1.5.17	movej_exj	52
3.1.5.18	movejp_exj	53
3.1.5.19	movec_exj	54
3.1.6	轨迹叠加功能函数	55
3.1.6.1	start_path_offset	55
3.1.6.2	end_path_offset	55
3.1.6.3	set_path_offset	55
3.1.6.4	get_path_offset	56
3.1.6.5	set_path_offset_limit	56
3.1.6.6	set_path_offset_filter	56
3.1.6.7	set_path_offset_speed	56
3.2	内部函数	57
3.2.1	时间函数	57
3.2.1.1	wait	57
3.2.1.2	get_time	57
3.2.1.3	get_systime	57
3.2.2	程序控制	58
3.2.2.1	task_create	58
3.2.2.2	task_pause	58
3.2.2.3	task_continue	59
3.2.2.4	task_delete	60

3.2.2.5 halt.....	61
3.2.2.6 pause.....	61
3.2.2.7 event_task_create.....	61
3.2.2.8 event_suspend.....	62
3.2.2.9 rt_event_register.....	63
3.2.2.10 rt_event_unregister.....	63
3.2.2.11 getlines.....	64
3.2.3 机器人系统函数.....	64
3.2.3.1 set_tcp.....	64
3.2.3.2 set_tcp_payload.....	64
3.2.3.3 get_tcp.....	65
3.2.3.4 get_target_tcp_pose.....	65
3.2.3.5 get_target_tcp_offset.....	66
3.2.3.6 get_move_target_tcp_pose.....	66
3.2.3.7 get_move_target_joint.....	66
3.2.3.8 get_tcp_force(coordinate).....	67
3.2.3.9 get_joint_torque_external().....	67
3.2.3.10 get_actual_tcp_pose.....	67
3.2.3.11 get_actual_tcp_speed.....	67
3.2.3.12 get_actual_joint_positions.....	68
3.2.3.13 get_actual_joint_speeds.....	68
3.2.3.14 get_actual_tcp_force.....	69
3.2.3.15 set_percentvelocity.....	69
3.2.3.16 get_percentvelocity.....	69
3.2.3.17 sync.....	70
3.2.3.18 get_actual_joint_torques.....	70
3.2.3.19 get_target_joint_torques.....	70
3.2.3.20 set_torprot_plus.....	71

3.2.3.21 set_mechanical_unit	71
3.2.3.22 set_wcs	71
3.3 计算函数	72
3.3.1 三角函数	72
3.3.1.1 sin	72
3.3.1.2 cos	72
3.3.1.3 tan	73
3.3.1.4 asin	73
3.3.1.5 acos	73
3.3.1.6 atan	74
3.3.1.7 atan2	74
3.3.2 位姿函数	74
3.3.2.1 pose_add	74
3.3.2.2 pose_sub	75
3.3.2.3 pose_trans	75
3.3.2.4 point_dist	76
3.3.2.5 pose_inv	76
3.3.2.6 pose_offset	77
3.3.2.7 euler2rotvec	79
3.3.2.8 rotvec2euler	79
3.3.2.9 inverse_kine_judge	79
3.3.2.10 inverse_kine	80
3.3.2.11 forward_kine	80
3.3.2.12 binary_list_to_integer	81
3.3.2.13 integer_to_binary_list	81
3.3.2.14 linear_interp_pose	81
3.3.3 数值运算函数	82
3.3.3.1 swap_bytes_for_word	82

3.3.3.2	swap_bytes_for_dword	82
3.3.3.3	swap_words_for_dword	83
3.3.3.4	commuvar_to_string	83
3.3.3.5	string_to_commuvar	83
3.3.4	位运算函数	84
3.3.4.1	bit_not	84
3.3.4.2	bit_and	84
3.3.4.3	bit_or	84
3.3.4.4	bit_xor	85
3.3.4.5	bit_lshift	85
3.3.4.6	bit_rshift	85
3.3.5	其他计算函数	86
3.3.5.1	ceil	86
3.3.5.2	floor	86
3.3.5.3	random	86
3.3.5.4	log	87
3.3.5.5	pow	87
3.3.5.6	sqrt	87
3.3.5.7	d2r	88
3.3.5.8	r2d	88
3.3.5.9	length	88
3.3.5.10	Lua math 库	89
3.4	输入输出函数	90
3.4.1	通讯变量	90
3.4.1.1	read_int16_register	90
3.4.1.2	read_integer_register	90
3.4.1.3	read_float_register	90
3.4.1.4	read_boolean_register	91

3.4.1.5 write_int16_register	91
3.4.1.6 write_integer_register	91
3.4.1.7 write_float_register	92
3.4.1.8 write_boolean_register	92
3.4.2 IO 变量	92
3.4.2.1 set_standard_digital_out	92
3.4.2.2 get_standard_digital_out	92
3.4.2.3 get_standard_digital_in	93
3.4.2.4 set_configurable_digital_out	93
3.4.2.5 get_configurable_digital_out	93
3.4.2.6 get_configurable_digital_in	94
3.4.2.7 set_standard_analog_out	94
3.4.2.8 get_standard_analog_in	94
3.4.2.9 set_tool_digital_out	95
3.4.2.10 get_tool_digital_in	95
3.4.2.11 get_tool_analog_in	95
3.4.2.12 get_tool_digital_out	96
3.4.2.13 ctrl_tool_24v_out	96
3.4.3 弹窗函数	96
3.4.3.1 str_cat	96
3.4.3.2 popup_message	97
3.4.3.3 popin_boolean	97
3.4.3.4 popin_number	97
3.4.3.5 popin_string	98
3.4.4 寄存器变量	98
3.4.4.1 get_sys_freq_float	98
3.4.4.2 get_sys_freq_int	98
3.4.4.3 get_sys_wreg_float	99

3.4.4.4 get_sys_wreg_short	99
3.4.4.5 get_sys_wreg_int	99
3.4.4.6 get_wreg_float	100
3.4.4.7 get_wreg_short	100
3.4.4.8 get_wreg_int	100
3.4.4.9 get_sys_hwreg_float	101
3.4.4.10 get_sys_hwreg_short	101
3.4.4.11 get_sys_hwreg_int	101
3.4.4.12 get_sys_breg	102
3.4.4.13 set_sys_freg_float	102
3.4.4.14 set_sys_freg_int	102
3.4.4.15 set_sys_freg_short	103
3.4.4.16 set_sys_wreg_float	103
3.4.4.17 set_sys_wreg_short	103
3.4.4.18 set_sys_wreg_int	104
3.4.4.19 set_wreg_float	104
3.4.4.20 set_wreg_short	105
3.4.4.21 set_wreg_int	105
3.4.4.22 set_sys_hwreg_float	105
3.4.4.23 set_sys_hwreg_short	106
3.4.4.24 set_sys_hwreg_int	106
3.4.4.25 set_sys_breg	106
3.5 通讯函数	107
3.5.1 Modbus 通讯	107
3.5.1.1 modbus_read_bits	107
3.5.1.2 modbus_read_input_bits	108
3.5.1.3 modbus_write_bit	108
3.5.1.4 modbus_write_bits	109

3.5.1.5 modbus_read_registers	109
3.5.1.6 modbus_read_input_registers	110
3.5.1.7 modbus_write_register	111
3.5.1.8 modbus_write_registers	111
3.5.2 网口通讯	112
3.5.2.1 socket_connect	112
3.5.2.2 socket_disconnect	113
3.5.2.3 socket_open	113
3.5.2.4 socket_close	113
3.5.2.5 socket_receive	113
3.5.2.6 socket_send	114
3.5.2.7 socket_receive_string	115
3.5.2.8 socket_send_string	116
3.5.2.9 socket_read_string	116
3.5.3 串口通讯	117
3.5.3.1 serial_receive	117
3.5.3.2 serial_send	117
3.5.4 QEP 正交脉冲计数	118
3.5.4.1 encoder_enable	118
3.5.4.2 encoder_disable	118
3.5.4.3 encoder_get_tick_count	118
3.5.4.4 encoder_get_strobe_tick_count	119
3.5.4.5 encoder_set_tick_count	119
3.5.5 通用通讯函数	119
3.5.5.1 get_last_receive_status	119
3.5.5.2 get_socket_connect_status	120
3.5.6 ethercat 通讯	120
3.5.6.1 read_ect_pdo	120

3.5.6.2 write_ect_pdo	120
3.5.6.3 read_ect_sdo	121
3.5.6.4 write_ect_sdo	121
3.5.7 辅助功能	122
3.5.7.1 read_masterboard_mb_reg	122

一、机器人脚本语言概述

1.1 脚本语言介绍

脚本编程是机器人的底层编程语言，用户可使用脚本生成自定义的命令。

1.2 Lua 简介

Lua 是一种轻量小巧的脚本语言，用标准 C 语言编写并以源代码形式开放，其设计目的是为了嵌入应用程序中，从而为应用程序提供灵活的扩展和定制功能。

1.3 编程环境及配置要求

脚本编程的应用场景包括：

- 示教器的脚本命令使用。
- 直接使用电脑端的脚本编辑器使用。

二、脚本语言编程基础

2.1 数据类型

数据类型见表 1。

表 1 数据类型

数据类型	描述
nil	这个最简单, 只有值 nil 属于该类, 表示一个无效值(在条件表达式中相当于 false)。例如: <code>type(X)=="nil"</code> 。
boolean	布尔型, 包含两个值: false 和 true。
number	数字, 表示双精度类型的实浮点数。
string	字符串由一对双引号或单引号来表示。
function	由 C 或 Lua 编写的函数。
userdata	表示任意存储在变量中的 C 数据结构。
thread	表示执行的独立线路, 用于执行协同程序。
table	Lua 中的表 (table) 其实是一个"关联数组" (associative arrays)。数组的索引可以是数字、字符串或表类型。在 Lua 里, table 的创建是通过"构造表达式"来完成。最简单构造表达式是 {}, 用来创建一个空表。

2.1.1 number 数字

Lua 默认只有一种 number 类型 double (双精度) 类型, 以下几种写法都被看作是 number 类型。

```
type(2)
type(2.2)
type(0.2)
type(2e+1)
type(0.2e-1)
type(7.8263692594256e-06)
```

2.1.2 string 类型

字符串由一对双引号或者单引号来表示, 也可以用 2 个方括号 "[[]]" 来表示 "一块" 字符串。

```
string1 = "this is string1"
string2 = 'this is string2'
html = [[
<html>
```

```
<head></head>

<body>

    <a href="http://www.runoob.com/">菜鸟教程</a>

</body>

<html>

]]

print(html)
```

使用#来计算字符串的长度，放在字符串前面。

```
len = "www.runoob.com"

print(#len)                --显示： 14

print("#www.runoob.com")   --显示： 14
```

2.1.3 table（表）

不同于其他语言的数组把 0 作为数组的初始索引，在 Lua 里表的默认初始索引一般以 1 开始。

在 Lua 里，table 的创建是通过“构造表达式”来完成，最简单构造表达式是 {}，用来创建一个空表。

也可以在表里添加一些数据，直接初始化表。

```
local tbl1 = {}                --创建一个空的 table

local tbl2 = {"apple", "pear", "orange", "grape"}   --直接初始表
```

Lua 中的表（table）的索引可以是数字或者是字符串。

```
a = {}

a["key"] = "value"

key = 10

a[key] = 22

a[key] = a[key] + 11

for k, v in pairs(a) do

    print(k .. " : " .. v)

end

--[[
```

输出结果为:

key : value

10 : 33

--]]

2.2 算术运算

算术运算操作符见表 2。

表 2 算术运算操作符

操作符	描述	实例, 设定 A 的值为 10, B 的值为 20
+	加法	A + B 输出结果 30
-	减法	A - B 输出结果-10
*	乘法	A * B 输出结果 200
/	除法	B / A 输出结果 2
%	取余	B % A 输出结果 0
^	乘幂	A^2 输出结果 100
-	负号	-A 输出结果-10

a = 21

b = 10

c = a + b

c = a - b

c = a * b

c = a / b

c = a % b

c = a^2

c = -a

2.3 位操作

Lua5.3 以后版本支持下列位操作符:

- &: 按位与。
- |: 按位或。
- ~: 按位异或。

- >>: 右移。
- <<: 左移。
- ~: 按位非。

2.4 注释

- 双短线为注释符号，在其后一行的内容即为注释内容。
- [[--]]双短线后加 2 个方括号为多行注释，方括号行之间的内容即为多行注释内容。

--单行注释的内容

--[[

多行注释的内容

多行注释的内容

--]]

备注：lua 多行注释时不支持嵌套注释。

--[[

多行注释的内容

--[[

多行注释的内容

多行注释的内容

--]]

此处的代码不属于注释内容，会执行

--]]

2.5 判断及关系运算符

判断及关系运算操作符见表 3。

表 3 判断及关系运算操作符

语句	描述
if 语句	if 语句由一个布尔表达式作为条件判断，其后紧跟其他语句组成。
if...else 语句	if 语句可以与 else 语句搭配使用，在 if 条件表达式为 false 时执行 else 语句代码。
if 嵌套语句	你可以在 if 或 elseif 中使用一个或多个 if 或 elseif 语句。

```
a = true

b = true

if ( a and b )

then

    print("a and b - 条件为 true" )

end

if ( a or b )

then

    print("a or b - 条件为 true" )

end

--修改 a 和 b 的值

a = false

b = true

if ( a and b )

then

    print("a and b - 条件为 true" )

else

    print("a and b - 条件为 false" )

end

if ( not( a and b ) )

then

    print("not( a and b ) - 条件为 false" )

end
```

```
--[ 检查布尔条件 --]

a = 100

if( a == 10 )
then
    print("a 的值为 10" )
elseif( a == 20 )
then
    print("a 的值为 20" )
elseif( a == 30 )
then
    print("a 的值为 30" )
else
    print("没有匹配 a 的值" )
end
```

判断及关系运算操作符见表 4。

表 4 判断及关系运算操作符

操作符	描述	实例
==	等于，检测两个值是否相等，相等返回 true，否则返回 false。	(A == B)为 false。
~=	不等于，检测两个值是否相等，相等返回 false，否则返回 true。	(A ~= B)为 true。
>	大于，如果左边的值大于右边的值，返回 true，否则返回 false。	(A > B)为 false。
<	小于，如果左边的值大于右边的值，返回 false，否则返回 true。	(A < B)为 true。
>=	大于等于，如果左边的值大于等于右边的值，返回 true，否则返回 false。	(A >= B)返回 false。
<=	小于等于，如果左边的值小于等于右边的值，返回 true，否则返回 false。	(A <= B)返回 true。
and	逻辑与操作符。若 A 为 false，则返回 A，否则返回 B。	(A and B)为 false。
or	逻辑或操作符。若 A 为 true，则返回 A，否则返回 B。	(A or B)为 true。
not	逻辑非操作符。与逻辑运算结果相反，如果条件为 true，逻辑非为 false。	not(A and B)为 true。

2.6 循环

循环操作符见表 5。

表 5 循环操作符

循环类型	描述
for 循环	重复执行指定语句，重复次数可在 for 语句中控制。
while 循环	在条件为 true 时，让程序重复地执行某些语句。执行语句前会先检查条件是否为 true。
repeat...until	重复执行循环，直到指定的条件为真时为止。
循环嵌套	可以在循环内嵌套一个或多个循环语句（while do ... end;for ... do ... end;repeat ... until;）。

循环操作符见表 6。

表 6 循环操作符

控制语句	描述
break 语句	退出当前循环或语句，并开始脚本执行紧接着的语句。
goto 语句	将程序的控制点转移到一个标签处。

2.6.1 for 循环

--var 从 exp1 变化到 exp2，每次变化以 exp3 为步长递增 var。exp3 是可选的，默认为 1

```
for var=exp1,exp2,exp3 do
```

```
    <执行体>
```

```
end
```

```
for i=1,10 do
```

```
    print(i)
```

```
end
```

```
for i=10,1,-1 do
```

```
    print(i)
```

```
end
```

```
array = {"Google", "Runoob"}
```

```
for key,value in ipairs(array)
```

```
do
```

```
    print(key, value)
```

```
end
```

2.6.2 while 循环

--condition 可以是任意表达式，在 condition 条件为 true 时执行循环体语句

```
while(condition)
do
    statements
end
```

```
a=10
while( a < 20 )
do
    print("a 的值为:", a)
    a = a+1
end
```

2.6.3 repeat 循环

--在条件进行判断前循环体都会执行一次

--如果条件判断语句（condition）为 false，循环会重新开始执行

--直到条件判断语句（condition）为 true 才会停止执行

```
repeat
    statements
until( condition )
```

```
a = 10
--[ 执行循环 --]
repeat
    a = a + 1
until( a > 15 )
```

2.6.4 break 语句

```
a = 10
while( a < 20 )
do
    print("a 的值为:", a)
    a=a+1
    if( a > 15)
    then
        break                --[使用 break 语句终止循环--]
    end
end
end
```

2.6.5 goto 和 label

Label 的格式为:

```
:: Label ::
goto Label
```

```
local a = 1
::labelA:: print("--- goto label ---")
a = a+1
if( a < 3 )
then
    goto labelA                -- a 小于 3 的时候跳转到标签 label
end
```

2.7 函数

在 Lua 中，函数是对语句和表达式进行抽象的主要方法。既可以用来处理一些特殊的工作，也可以用来计算一些值。

2.7.1 函数定义

Lua 编程语言函数定义格式如下。

```
optional_function_scope function function_name( argument1, argument2, argument3..., argumentn)

    function_body

    return result_params_comma_separated

end
```

函数可以作为参数传递。

```
myprint = function(param)

    print("这是打印函数 -   ##",param,"##")

end

function add(num1,num2,functionPrint)

    result = num1 + num2

    --调用传递的函数参数

    functionPrint(result)

end

myprint(10)

--myprint 函数作为参数传递

add(2,5,myprint)
```

2.7.2 多返回值

Lua 函数可以返回多个结果值，在 `return` 后列出要返回的值的列表即可返回多值，如果不存在返回 `nil`。

```
function maximum (a)

    local mi = 1                --最大值索引

    local m = a[mi]             --最大值

    for i,val in ipairs(a) do

        if val > m then

            mi = i

        end

    end

    return m
```

```

        m = val
    end
end
return m, mi                --返回两个值
end
print(maximum({8,10,23,12,5}))

```

2.7.3 可变参数

Lua 函数可以接受可变数目的参数，和 C 语言类似，在函数参数列表中使用三点...表示函数有可变的参数。

- ...表示函数有可变的参数列表。
- 也可以通过 `select("#",...)` 来获取可变参数的数量。

```

function average(...)
    result = 0
    local arg={...}
    for i,v in ipairs(arg) do
        result = result + v
    end
    print("总共传入 " .. select("#",...) .. " 个数")
    return result/select("#",...)
end
print("平均值为",average(10,5,3,4,5,6))

```

有时候我们可能需要几个固定参数加上可变参数，固定参数必须放在变长参数之前。

```

function fwrite(fmt, ...)                --&gt; 固定的参数 fmt
    return io.write(string.format(fmt, ...))
end

fwrite("runoob\n")                        --&gt; fmt = "runoob", 没有变长参数
fwrite("%d%d\n", 1, 2)                    --&gt; fmt = "%d%d", 变长参数为 1 和 2

```

通常在遍历变长参数的时候只需要使用 `{...}`，然而变长参数可能会包含一些 `nil`，那么就可以用

select 函数来访问变长参数了：select('#', ...) 或者 select(n, ...)。

- select('#', ...) 返回可变参数的长度。
- select(n, ...) 用于访问 n 到 select('#',...) 的参数。

调用 select 时，必须传入一个固定实参 selector(选择开关)和一系列变长参数。如果 selector 为数字 n, 那么 select 返回它的第 n 个可变实参，否则只能为字符串"#",这样 select 会返回变长参数的总数。

```
do
    function foo(...)
        for i = 1, select('#', ...) do          --&gt;获取参数总数
            local arg = select(i, ...);        --&gt;读取参数
            print("arg", arg);
        end
    end
end
foo(1, 2, 3, 4);
end
```

2.8 常用函数

2.8.1 print

```
print(a[1], ",", a[2], ",", a[3])

a=123                                --数值
b="c123"                             --字符串
c={1,2,3}                            --数组

print(a)
print(b)

print(#b)                            --#用于计算长度
print(#c)

print(c[1])

print(c)

for i=1,3 do
```

```
print(c[i])  
end
```

三、函数分类

3.1 运动函数

3.1.1 基础运动函数

3.1.1.1 movej

描述

关节运动至目标关节角度。

参数

double angle[6]: 目标点关节角度(单位为度);

double jointSpeed: 关节角速度(单位为度每秒);

double jointAcc: 关节角加速度(单位为度每秒的平方);

double blendRadius: 过渡半径(单位为 mm)。

示例

```
w_1_p={-444.417,-146.043,538.816,118.126,-125.968,-8.70271}
```

```
w_1_q={4.04722,-6.11458,76.4685,10.5067,-89.9962,7.72768}
```

```
w_2_p={-541.663,-69.2794,410.851,106.144,-138.147,-8.08259}
```

```
w_2_q={-4.76532,5.55084,98.0186,-21.2833,-90.1573,10.1506}
```

```
movej(w_1_q,60,80,10)
```

```
movej(w_2_q,60,80,10)
```

--定义两个目标点相对于基坐标系的坐标值和目标点参考关节角度(单位为度)，关节角速度为 60°/s,加速度为 80°/s²，过渡半径为 10mm。

3.1.1.2 movejp

描述

关节运动至目标坐标（相对于基坐标系）。

参数

double pose[6]: 目标点相对于基坐标系的坐标值(数据格式{x,y,z,rx,ry,rz},rx,ry,rz 以轴角的方式描述);

double refAngle[6]: 目标点参考关节角度(单位为度)，注：可省略；

double jointSpeed: 关节角速度(单位为度每秒);

double jointAcc: 关节角加速度(单位为度每秒的平方);

double blendRadius: 过渡半径(单位为 mm)。

示例

```
w_1_p={-444.417,-146.043,538.816,118.126,-125.968,-8.70271}
w_1_q={4.04722,-6.11458,76.4685,10.5067,-89.9962,7.72768}
w_2_p={-541.663,-69.2794,410.851,106.144,-138.147,-8.08259}
w_2_q={-4.76532,5.55084,98.0186,-21.2833,-90.1573,10.1506}
movejp(w_1_p,w_1_q,60,80,10)
movejp(w_2_p,w_2_q,60,80,10)
```

--定义两个目标点相对于基坐标系的坐标值和目标点参考关节角度(单位为度)，关节角速度为 60°/s,加速度为 80°/s²，过渡半径为 10mm

3.1.1.3 movel

描述

直线运动至目标坐标（相对于基坐标系）。

参数

double pose[6]: 目标点相对于基坐标系的坐标值(数据格式 {x,y,z,rx,ry,rz},rx,ry,rz 以轴角的方式描述);

double refAngle[6]: 目标点参考关节角度(单位为度)，注：可省略；

double Speed: 运动线速度(单位为毫米每秒);

double Acc: 运动线加速度(单位为毫米每秒的平方);

double blendRadius: 过渡半径(单位为 mm)。

示例

```
w_1_p={-444.417,-146.043,538.816,118.126,-125.968,-8.70271}
w_1_q={4.04722,-6.11458,76.4685,10.5067,-89.9962,7.72768}
w_2_p={-541.663,-69.2794,410.851,106.144,-138.147,-8.08259}
w_2_q={-4.76532,5.55084,98.0186,-21.2833,-90.1573,10.1506}
movel(w_1_p,60,80,10)
movel(w_2_p,60,80,10)
```

--定义两个目标点相对于基坐标系的坐标值和目标点参考关节角度(单位为度)，关节角速度为

60°/s,加速度为 80°/s², 过渡半径为 10mm

3.1.1.4 movec

描述

圆弧运动至目标坐标（相对于基坐标系）。

参数

double pose_mid[6]: 圆弧中间点相对于基坐标系的坐标值（数据格式 {x,y,z,rx,ry,rz},rx,ry,rz 以轴角的方式描述）；

double refAngle_mid[6]: 圆弧中间点参考关节角度(单位为度)，注：可省略；

double pose_end[6]: 圆弧目标点相对于基坐标系的坐标值（数据格式 {x,y,z,rx,ry,rz},rx,ry,rz 以轴角的方式描述）；

double refAngle_end[6]: 圆弧目标点参考关节角度(单位为度)，注：可省略；

double Speed: 运动线速度(单位为毫米每秒)；

double Acc: 运动线加速度(单位为毫米每秒的平方)；

double blendRadius: 过渡半径(单位为 mm)；

int fixedRot: 非零则位姿绕圆的轴线旋转，注：可省略；

double arcAngle: 圆弧角(单位为度)，注：可省略。

示例

```
w_1_p={114.3,-492,490.9,-180,-1.3411e-6,8.15762e-6}           --圆弧初始点 TCP 姿态
w_1_q={90,0,90,0,-90,0}                                       --圆弧初始点关节角度
w_2_p={322.665,-405.526,490.898,-180,-1.3411e-6,1.01251e-5}   --圆弧中间点 TCP 姿态
w_2_q={115.766,1.81618,88.1529,0.030899,-90,25.7664}         --圆弧中间点关节角度
w_3_p={498.146,-245.033,490.906,180,0,0.000840388}           --圆弧目标点 TCP 姿态
w_3_q={141.926,6.92825,82.6186,0.452499,-90,51.9262}         --圆弧目标点关节角度
movejp(w_1_p,w_1_q,60,80,0)                                     --移动到初始点
movec(w_2_p,w_2_q,w_3_p,w_3_q,250,1200,25,0)
--运动线速度为 250mm/s, 线加速度为 1200mm/s2, 过度半径为 25mm
```

3.1.1.5 addwaypoint

描述

增加 movex 指令的路点，在 movex 指令前配合 movex 使用。

参数

double pose[6]: 目标点相对于基坐标系的坐标值(数据格式 {x,y,z,rx,ry,rz}, rx,ry,rz 以轴角的方式描述)。

示例

```
waypoint_1_p={649.73,-101.579,424.097,168.3,63.8341,0.006853}    --点位 1TCP 姿态
addwaypoint(waypoint_1_p)    --增加点位 1 路点
```

3.1.1.6 movex

描述

多个路点匀速运动，自动按照过渡半径来进行交融；在执行之前，需要使用 addwaypoint 指令添加路点。

参数

double blendRadius: 过渡半径(单位为 mm)；

double Speed: 运动线速度(单位为毫米每秒)；

double Acc: 运动线加速度(单位为毫米每秒的平方)。

示例

```
waypoint_1_p={649.73,-101.579,424.097,168.3,63.8341,0.006853}    --点位 1TCP 姿态
waypoint_2_p={654.742,-96.5788,424.104,168.294,63.8407,0.0142932}    --点位 2TCP 姿态
waypoint_3_p={659.741,-91.5759,424.111,168.292,63.8464,0.0153429}    --点位 3TCP 姿态
addwaypoint(waypoint_1_p)    --增加点位 1 路点
addwaypoint(waypoint_2_p)    --增加点位 2 路点
addwaypoint(waypoint_3_p)    --增加点位 3 路点
movex(50,200,5000)    --过渡半径为 50mm,运动线速度为 200mm/s,线加速度为 5000mm/s²
```

3.1.1.7 moveps

描述

笛卡尔空间多路点运动，按照直线段+曲线过渡方式运动，过渡半径可设置。使用 moveps 后，控制器等待上层添加路点，当添加的路点数目达到 segment 个时，机械臂开始运动，之后控制器边运动边处理上层下发的路点，直到收到结束信号。

参数

double speed: 运动线速度（单位：mm/s）；

double acceleration: 运动线加速度（单位：mm/s²）；

double blendR: 过渡半径（单位：mm）；

int segment: 分段数目（个），默认大于等于 3，当下发的路点个数超过 segment 时，控制器才会处理这些路点。若下发的路点不足 segment 个，控制器会等待路点下发直到路点数目超过 segment，或者等待 moveps_end 信号，这样才继续处理这些数据。

示例

```
movejp({-500,-200,500,-120,120,0},60,80,0)
moveps(200,500,10,3)
--分段多点运动，速度 200mm/s，加速度 500mm/s2，过渡半径 10mm，分段数目 3
addwaypoint({-500,-200,500,-120,120,0})
addwaypoint({-400,-310,410,-110,130,0})
addwaypoint({-300,-200,520,-100,140,0})
addwaypoint({-200,-410,530,-120,120,0})
addwaypoint({-300,-220,440,-110,100,0})
addwaypoint({-200,-410,530,-120,120,0})
addwaypoint({-500,-210,560,-100,120,0})    --添加路点
moveps_end()    --结束分段多点运动
```

注：moveps 和 moveps_end 需要成对使用，中间除了添加路点的指令之外不能添加其他运动指令；过渡半径只是参考值，真实的过渡半径还受到线段长度的影响；允许有折返、过渡半径为 0 的情况，但会导致速度不连续，可能会造成抖动；若上层添加路点不及时，即走完已经添加的路点仍没有新的路点下发，控制器将暂停运动并等待路点下发；调用 moveps 后会默认添加一个路点，所以第一段轨迹在添加了 segment-1 个路点后会开始执行。

3.1.1.8 moveps_end

描述

结束分段多点运动。

参数

无。

示例

```

movejp({-500,-200,500,-120,120,0},60,80,0)

moveps(200,500,10,3)

--分段多点运动，速度 200mm/s，加速度 500mm/s^2，过渡半径 10mm，分段数目 3

addwaypoint({-500,-200,500,-120,120,0})

addwaypoint({-400,-310,410,-110,130,0})

addwaypoint({-300,-200,520,-100,140,0})

addwaypoint({-200,-410,530,-120,120,0})

addwaypoint({-300,-220,440,-110,100,0})

addwaypoint({-200,-410,530,-120,120,0})

addwaypoint({-500,-210,560,-100,120,0})      --添加路点

moveps_end()      --结束分段多点运动

```

3.1.1.9 movestop

描述

停止运动指令，在不同于运动任务的另外一个任务使用，指令执行后，运动停止，并且程序在停止完成后跳转至下一行程序。

参数

int stop_type: 大于 0 时按当前加速度停止；等于 0 时按最大加速度停止；小于 0 时执行紧急停止。

示例

```
movestop(0)
```

3.1.1.10 set_freedrive

描述

设置自由驱动启动，开启后机械臂可进行自由驱动。

参数

bool FreeDriveEnabled (true false)。

示例

```
set_freedrive(true)
```

3.1.1.11 get_freedrive

描述

获取是否为自由驱动状态。

参数

无。

返回值

bool res: 自由驱动状态，true 为自由驱动，false 为非自由驱动。

示例

```
res=get_freedrive()
```

3.1.1.12 set_jerk_acc_ratio

描述

设定加加速度与加速度的比例。

参数

double ratio: 加加速度与加速度的比例。

返回值

无。

示例

```
set_jerk_acc_ratio(2)      --加加速度的值为加速度的 2 倍
```

3.1.2 传送带功能函数

3.1.2.1 conv_deact

描述

在运动停止状态下去激活传送带。

参数

int id: 16 位有效的传送带编号。

示例

```
conv_deact(1)      --激活编号为 1 的传送带
```

3.1.2.2 conv_act

描述

在运动停止状态下对非激活的传送带进行配置并激活。

参数

int id: 16 位有效的传送带编号;

int chnn: 传送带编码器通道选择, 取值范围为 0~1 (若>1 则默认为 1);

double conv_pose[6]或 double conv_dir[3]: 前者为传送带坐标系到基坐标系变换对应的位姿描述, 其 +X 轴为传送带移动正方向, 后者则为在基坐标系描述的传送带移动正方向矢量 (模可以不等于 1);

double enc_resol: 编码器分辨率, 单位是脉冲数/mm;

double work_lim: 传送带抓取窗口下限, 含义是基坐标系原点到目标的矢量在传送带方向的投影。

示例

```
conv_pose = {-492,-314.3,490.901,0.000216694,1.3442e-5,90.0001}
--传送带坐标系到基坐标系变换对应的位姿描述

conv_act(1,1,conv_pose,13.1*40,500)
--编号为 1 的传送带, 通道为 1, 编码器分辨率为 13.1*40, 传送带抓取窗口下限为 500

conv_act(1,1,{0.0,1.0,0.0},13.1*40,500)
--传送带坐标到基坐标变换为{0.0, 1.0, 0.0}, 其他同上
```

3.1.2.3 start_conv_track

描述

使用可配置的最大速度和加速度参数, 直线跟随传送带上的动目标, 编码器数值和目标位姿严格时间同步;

跟随成功后, 可以 wait 时间或信号, 或运行同步关节、直线和圆轨迹, 或者直接 stop_conv_track 停止退出;

可以直接使用"编程->基础->移动/圆弧移动"或"movej/movejp/movel /movec"脚本来实现关节和各种参考坐标系位姿的同步轨迹, 使用方式与非传送带模式无差别, 但对应的关节或位姿与 start_conv_track 编码器 enc_sync 时间同步; 如果想使用脚本实现同步轨迹, 但又不想手动转换工件坐标系位姿到基坐标系, 则可以使用 movel_wobj 和 movec_wobj 专用脚本指令。

参数

int id: 16 位有效的传送带编号;

double pose[6]: 目标位姿 (数据格式{x,y,z,rx,ry,rz},rx,ry,rz 以轴角的方式描述);

double coord_pose[6]: 目标位姿位姿所对应的参考坐标系（参考系到基坐标系变换），格式同 pose;

int enc_sync: 与目标位姿时间同步的编码器数值;

double vm: 跟随最大的线速度（相对于传送带运动），单位 mm/s;

double am: 跟随最大的线加速度（相对于传送带运动），单位 mm/s²。

示例

```
start_conv_track(1,{-492,-414.3,490.9,127.28,-127.28,0},{0,0,0,0,0,0},25839,2000,10000)

--传送带编号为 1，目标位姿为{-492,-414.3,490.9,127.28,-127.28,0}，对应的参考坐标系为
{0,0,0,0,0,0}，目标位姿时间同步的编码器数值为 25839，跟随最大线速度为 2000mm/s,跟随最大
的线加速度为 10000mm/s2
```

3.1.2.4 stop_conv_track

描述

传送带跟随停止，关节运动减速停止到固定目标关节或固定基坐标系位姿（类似 movej/movejp），或沿传送带方向减速停止。

参数格式一

double pose[6]: 基坐标系目标点位姿（数据格式 {x,y,z,rx,ry,rz},rx,ry,rz 以轴角的方式描述），可缺省;

double angle[6]: 当位姿缺省时，为目标点关节角度(单位为度)，否则，为位姿参考关节;

double jointSpeed: 关节角速度(单位为度每秒);

double jointAcc: 关节角加速度(单位为度每秒的平方);

double blendRadius: 过渡半径(单位为 mm)。

参数格式二

int stop_type: 大于等于 0 时，传送带方向减速停止，否则，立即停止。

示例

```
stop_conv_track({-10,-45,90,0,-90,0},200,2000,10)

--格式一：位姿缺省，目标点关节角度为{-10,-45,90,0,-90,0}，角速度为 200°/s，角加速度为 2000°/s2，
过渡半径为 10mm

stop_conv_track(0)                                --格式二：参数为 0，减速停止。
```

3.1.2.5 movel_wobj

描述

传送带同步直线轨迹，与 `move1` 主要区别在于可以使用工件坐标系位姿，暂不支持参考关节。

参数

`double pose[6]`: 目标点位姿（数据格式 `{x,y,z,rx,ry,rz}`, `rx,ry,rz` 以轴角的方式描述），与 `start_conv_track` 编码器 `enc_sync` 时间同步；

`double conv_pose[6]`: 位姿参考坐标系，即工件系到基坐标系变换，格式同 `pose`；

`double Speed`: 运动线速度(单位为毫米每秒)；

`double Acc`: 运动线加速度(单位为毫米每秒的平方)；

`double blendRadius`: 过渡半径(单位为 mm)。

示例

```
conv_pose = {-492,-314.3,490.901,0.000216694,1.3442e-5,90.0001}           --位姿参考坐标系
move1_wobj({-100,-200,50,0,180,0}, conv_pose, 200, 1000, 20)
--目标点位姿为{-100,-200,50,0,180,0}，位姿参考坐标系为 conv_pose，运动线速度为 200mm/s，
运动线加速度为 1000mm/s²，过渡半径为 20mm
```

3.1.2.6 movec_wobj

描述

传送带同步圆弧轨迹，与 `movec` 主要区别在于可以使用工件坐标系位姿，暂不支持参考关节。

参数

`double pose_mid[6]`: 圆弧中间点位姿（数据格式 `{x,y,z,rx,ry,rz}`, `rx,ry,rz` 以轴角的方式描述），与 `start_conv_track` 编码器 `enc_sync` 时间同步；

`double pose_end[6]`: 圆弧目标点位姿（数据格式 `{x,y,z,rx,ry,rz}`, `rx,ry,rz` 以轴角的方式描述），与 `start_conv_track` 编码器 `enc_sync` 时间同步；

`double conv_pose[6]`: 位姿参考坐标系，即工件系到基坐标系变换，格式同 `pose`；

`double Speed`: 运动线速度(单位为毫米每秒)；

`double Acc`: 运动线加速度(单位为毫米每秒的平方)；

`double blendRadius`: 过渡半径(单位为 mm)；

`int fixedRot`: 非零则位姿绕圆的轴线旋转。

示例

```
conv_pose = {-492,-314.3,490.901,0.000216694,1.3442e-5,90.0001}           --位姿参考坐标系
```

```
movec_wobj({-100,0,50,0,180,0},{-100,-200,50,0,180,0},conv_pose,500,1000,20,0)
```

```
--圆弧中间点位姿为{-100,0,50,0,180,0}，圆弧目标点位姿{-100,-200,50,0,180,0}
```

3.1.3 力控碰撞功能函数

3.1.3.1 start_force_mode

描述

启动力控，由配置的参数决定机械臂相应自由度的柔性或者出力状态。

参数

table task_frame: task_frame[1]为 0 代表力控坐标系是相对机器人基坐标系是固定的，1 代表动坐标系；task_frame[2]为力控坐标系，长度为 6 的 table；

bool flexible_axis[6]: 柔性轴，长度为 6 的 table；true 为柔性轴，false 非柔性轴；

double Wrench[6]: 六维力/力矩，长度为 6 的 table；

double trans_frame[6]: 传感器坐标系(法兰坐标系下表达)，长度为 6 的 table；

double Limits(可缺省): 力控限制(柔性轴为速度限制，非柔性轴为位置偏差限制)，长度为 6 的 table。

示例

(1)

```
task_frame={0,{0,0,0,0,0,0}}           --力控坐标系为基坐标系，静止坐标系
axis={false,false,true,false,false,false}  --z 轴为柔性轴，x,y,Rx,Ry,Rz 为非柔性轴
Wrench={0,0,-10,0,0,0}      --z 轴负向施加 10N 的力(因为力控坐标系为基坐标系，所以竖直向下)
trans_frame={0,0,0,0,0,0}      --传感器坐标系和法兰坐标系一致
start_force_mode(task_frame,axis,Wrench,trans_frame) --注释如上
```

(2)

```
task_frame={1,{100,0,0,0,0,0}}          --力控坐标系为{100,0,0,0,0,0}，动坐标系
axis={true,true,false,false,false,false} --xoy 平面为柔性平面，z,Rx,Ry,Rz 为非柔性轴
Wrench={0,0,0,0,0,0}
trans_frame={0,0,0,0,0,-45}              --绕法兰坐标系 z 轴旋转-45 为力传感器坐标系
limits={0.1,0.1,0.005,0.002,0.002,0.002}
--x,y 方向速度保护为 0.1m/s，z 方向偏差保护为 0.005m,Rx,Ry,Rz 偏差保护为 0.002rad
```

```
start_force_mode(task_frame,axis,Wrench,trans_frame,limits)
```

3.1.3.2 end_force_mode

描述

关闭力控，机械臂的所有自由度重新恢复刚性。

参数

int type: 关闭力控的方式。未填写参数时，关闭力控会停止机械臂运动并停止程序；参数为 1 时，关闭力控时力控偏差回退，不停止运动和程序；参数为 2 时，关闭力控时力控偏差保持原位，停止机械臂运动，不停止程序，该关闭力控的方式要求调用时不能正在进行或者将要进行机械臂运动，否则将会切换到 end_force_mode() 的方式，在 end_force_mode(2) 完成后可以继续调用运动函数。

注：当已经开启了 end_force_mode(2) 并开始进入关闭力控的过程中时，不允许再调用 end_force_mode(1)；只允许 end_force_mode(2) 单向切换到 end_force_mode()，以及 end_force_mode(1) 单向切换到 end_force_mode()。此函数为阻塞式函数。

示例

```
end_force_mode() --关闭力控模式
```

3.1.3.3 force_mode_payload

描述

根据负载质量和重心对六维力/力矩传感器采到的值进行补偿，剔除负载的影响，保证用于控制的力为外部力。

参数

double mass: 负载质量，有效值 0~10（单位：kg）。

double xyz[3]: 负载重心（在法兰坐标系下的表达，单位：mm）。

示例

```
force_mode_payload(0.1,{0,0,100})
--传感器所带负载 0.1kg,负载重心沿着法兰 Z 轴正向 100mm 处
```

注：质量和重心设置不正确会导致没有外力作用的情况下轨迹偏离；后期会开发质量和重心的标定算法。

3.1.3.4 force_mode_mass

描述

和物体的质量相似的性质，代表物体的惰性，惯性越大，相同外力作用下加速度越小，在机器人力控制中设置大的惯性参数有利于机器人的稳定性，但是惯性参数过大机器人的柔性便相对减弱；惯性参数太小会造成抖动，以及和刚性物体接触出现跳动等问题。

参数

double massPara: 质量系数，为 number 或长度为 6 的 table，有效值 0.5~10。

示例

- | | |
|--------------------------------------|---------------------------------|
| (1) force_mode_mass(1) | --x,y,z,Rx,Ry,Rz 质量均为 1 |
| (2) force_mode_mass({1,1,0.5,1,1,1}) | --x,y,Rx,Ry,Rz 质量均为 1,z 质量为 0.5 |

注：质量设置的过小可能会产生较大的加速度，如非必要不随便更改。

3.1.3.5 force_mode_damping

描述

和运动物体受到摩擦相似的性质，摩擦参数增大有利于机器人的稳定性，减小震荡，但是摩擦参数设置过大机器人的柔性便相对减弱，运动速度可能变缓；同样阻尼系数设置太小有可能会造成机器人的抖动。

参数

double dampingPara: 阻尼系数，为 number 或长度为 6 的 table，有效值 0.1~10。

示例

- | | |
|---|---------------------------------|
| (1) force_mode_damping(0.1) | --x,y,z,Rx,Ry,Rz 阻尼均为 0.1 |
| (2) force_mode_damping({1,1,0.1,1,1,1}) | --x,y,Rx,Ry,Rz 阻尼均为 1,z 阻尼为 0.1 |

3.1.3.6 force_mode_stiffness

描述

和弹簧的弹性相似的性质，增大刚度系数，机器人可以更加有效的跟随轨迹，但是在和刚性物体接触时，过大的刚性可能造成机器人损坏；刚度系数设的太小可能导致轨迹的跟随严重滞后，刚度系数为 0 则该轴的运动特性完全丢失成为自由柔性轴。

参数

stiffnessPara: 刚度系数，为 number 或长度为 6 的 table，有效值 0~10。

示例

- | | |
|-------------------------------|---------------------------|
| (1) force_mode_stiffness(0.1) | --x,y,z,Rx,Ry,Rz 刚度均为 0.1 |
|-------------------------------|---------------------------|

(2) force_mode_stiffness({1,1,0.1,1,1,1})

--x,y,Rx,Ry,Rz 刚度均为 1,z 刚度为 0.1

3.1.3.7 start_freedrive_mode

描述

启动力控拖动，由配置的参数决定机械臂相应自由度可拖动状态。

参数

table task_frame: task_frame[1]为 0 代表力控坐标系是相对机器人基坐标系是固定的，1 代表动坐标系；task_frame[2]为力控坐标系，长度为 6 的 table；

bool axis[6]: 可拖动轴，缺省情况下所有轴可拖动；

double trans_frame[6]: 传感器坐标系(法兰坐标系下表达)，长度为 6 的 table。

示例

(1)

task_frame={1,{100,0,0,0,0,0}}

--力控坐标系为{100,0,0,0,0,0}，动坐标系

axis={false,false,true,false,false,false}

--z 轴可拖动

trans_frame={0,0,0,0,0,0}

--传感器坐标系和法兰坐标系一致

start_freedrive_mode(task_frame,axis,trans_frame)

--注释如上

(2)

task_frame={0,{0,0,0,0,0,0}}

--力控坐标系为基坐标系，静止坐标系

trans_frame={0,0,0,0,0,-45}

--绕法兰坐标系 z 轴旋转-45 为力传感器坐标系

start_freedrive_mode(task_frame,trans_frame)

--所有轴可拖动(和设置的力控坐标系没有关系)

3.1.3.8 end_freedrive_mode

描述

关闭力控拖动，机械臂的所有自由度重新恢复刚性。

参数

int type: 关闭力控的方式。未填写参数时，关闭力控会停止机械臂运动并停止程序；参数为 1 时，关闭力控时力控偏差回退，不停止运动和程序；参数为 2 时，关闭力控时力控偏差保持原位，停止机械臂运动，不停止程序，该关闭力控的方式要求调用时不能正在进行或者将要进行机械臂运动，否则将会切换到 end_freedrive_mode()的方式，在 end_freedrive_mode(2)完成后可以继续调用运动函数。

注：当已经开启了 end_freedrive_mode(2)并开始进入关闭力控的过程中时，不允许再调用

end_freedrive_mode(1)；只允许 end_freedrive_mode(2) 单向切换到 end_freedrive_mode()，以及 end_freedrive_mode(1) 单向切换到 end_freedrive_mode()。此函数为阻塞式函数。

示例

```
end_freedrive_mode()           --关闭力控拖动
```

3.1.3.9 get_force_mode_wrench

描述

获取传感器采集到的六维力/力矩（去除负载重力/力矩）。

参数

无。

返回值

double wrench[6]。

示例

```
wrench=get_force_mode_wrench()   --获取六维力矩
```

注：得到的六维力/力矩值已经扣除负载部分，相当于外界作用力，坐标系为法兰坐标系。

3.1.3.10 is_force_mode

描述

获取当前力控状态，是否处于力控模式。

参数

无。

示例

```
a = is_force_mode()           --获取当前力控状态，赋值到 a（布尔）
```

3.1.3.11 force_mode_tcp

描述

力控 tcp 下的力控功能。

参数

double tcp[6]：力控 tcp（数据格式 {x,y,z,rx,ry,rz}，x, y, z 为位置，单位为 mm，rx, ry, rz 为姿态，以轴角的方式描述，单位为°）；

示例

```
tcp={0,0,200,0,0,10}
force_mode_tcp(tcp)
```

3.1.3.12 set_collision_level

描述

参数可以是 number 或者 table，有效值为 1-100(1 为最大)，参数为 number 则六个关节一致，table 则六个关节对应 table 的六个值。

参数

double Level/Level[6]：碰撞等级。

示例

```
set_collision_level(100)           --六个关节碰撞检测等级均为 100
a={10,20,30,40,50,60}             --创建一个表
set_collision_level(a)             --六个关节的碰撞检测等级分别对应 table 内六个数值
```

3.1.3.13 set_limits_type

描述

切换碰撞灵敏度等级。

参数

int Level：碰撞灵敏度等级，范围 1~5，其中 1 对应限制最多，2 对应限制偏多，3 对应限制偏少，4 对应限制最少，5 对应关闭碰撞检测；若当前碰撞检测为关闭，可通过设置为 1~4 中任意值打开碰撞检测。注：关闭碰撞检测可能存在很大的安全风险，请谨慎操作。

示例

```
set_limits_type(1)
```

3.1.4 轨迹再现功能函数

3.1.4.1 moves

描述

以样条曲线的形式复现轨迹，需先保存轨迹数据。

参数

int pathIndex：轨迹索引，取值范围 0~49；

double mvTime：相邻轨迹点之间的执行周期时间，取值范围 10~10000ms；

int jointMode: 是否使用关节复现, 0-不使用, 1-使用, 注: 可省略;

int ioNum: 复现的 IO 数量, 注: 可省略;

int ioType[]: 复现的 IO 类型, 0-标准数字输出/1-可配置数字输出/2-工具端数字输出/3-模拟量输出/4-扩展 IO 输出, 注: 可省略;

int ioIndex[]: 复现的 IO 索引, 注: 可省略;

int eTcp: 是否为外部 TCP, 0 表示非外部 TCP, 非 0 表示外部 TCP, 注: 可省略;

double[6] toolPos: 工具末端相对于工具法兰的偏移向量, 注: 可省略;

double[6] wobjPose: 工作坐标系相对于基坐标系的偏移向量, 注: 可省略。

示例

```
moves(5,100)                --复现索引为 5 的轨迹, 相邻轨迹点之间的执行周期为 100ms

firstPoint = get_first_wp(0) --获取轨迹 0 的第一个点

--TCP 复现

movejp({firstPoint[1],firstPoint[2],firstPoint[3],firstPoint[4],firstPoint[5],firstPoint[6]},60,80,0)

--movejp 移动到轨迹第一个 TCP 位置

moves(0,100) --以 TCP 复现方式复现 TCP 轨迹

--关节复现

movej({firstPoint[7],firstPoint[8],firstPoint[9],firstPoint[10],firstPoint[11],firstPoint[12]},60,80,0)

--movej 移动到轨迹第一个关节位置

moves(0,100,1) --以关节复现方式复现关节轨迹

--关节复现+IO 复现

movej({firstPoint[7],firstPoint[8],firstPoint[9],firstPoint[10],firstPoint[11],firstPoint[12]},60,80,0)

--movej 移动到轨迹第一个关节位置

moves(0,100,1,2,{0,0},{0,1}) --以关节复现方式复现关节轨迹和 IO 信号 (DO0, DO1)
```

3.1.4.2 check_index

描述

检查该索引下是否已经有轨迹数据。

参数

int pathIndex: 轨迹索引, 取值范围-1~49。

返回值

int res: 是否存在数据, 0 表示不存在, 1 表示存在。

示例

```
res= check_index(5)  --判断索引为 5 的轨迹是否存在数据, res 为 0 表示不存在, 为 1 表示存在
```

3.1.4.3 start_record

描述

开始记录轨迹, 默认采样周期 100ms。

参数

int ioNum: 记录的 IO 数量, 注: 可省略;

int ioType[]: 记录的 IO 类型, 0-标准数字输出/1-可配置数字输出/2-工具端数字输出/3-模拟量输出/4-扩展 IO 输出/5-标准数字输入/6-可配置数字输入/7-工具端数字输入/8-模拟量输入/9-工具端模拟量输入/10-扩展 IO 输入, 注: 可省略;

int ioIndex[]: 记录的 IO 索引, 注: 可省略。

返回值

无。

示例

```
(1) start_record()  --开始记录轨迹, 只记录轨迹
```

```
(2) start_record(3,{5,5,6},{0,1,0})
```

```
--开始记录轨迹, 记录轨迹同时记录三个 IO 信号, 分别为 DI0、DI1 和 DI2
```

3.1.4.4 stop_record

描述

停止记录轨迹。

参数

无。

返回值

无。

示例

```
stop_record()
```

--停止记录轨迹

3.1.4.5 path_save

描述

存轨迹到 flash，断电保存。

参数

int pathIndex: 轨迹索引，取值范围 0~49。

返回值

无。

示例

```
path_save(5)
```

--保存轨迹为索引值 5

3.1.4.6 get_num_of_wp

描述

实时获取当前已记录的轨迹点数。

参数

无。

返回值

int pointsNum: 实时获取的当前已记录的轨迹点数。

示例

```
pointsNum=get_num_of_wp()
```

3.1.4.7 get_first_wp

描述

获取当前索引轨迹的第一个点。

参数

int pathIndex: 轨迹索引，取值范围 0-49。

返回值

double[12] point: 轨迹点对应的笛卡尔空间位置(mm)和关节角度(°)。

示例

```
firstPoint=get_first_wp(5)           --firstPoint={x,y,z,rx,ry,rz,j1,j2,j3,j4,j5,j6}
```

3.1.4.8 get_path_play_index

描述

实时获取当前已复现的轨迹点数（指轨迹复现时，实时监控当前跑到第几个点）。

参数

无。

返回值

int pointIndex: 当前执行到的轨迹点索引。

示例

```
pointIndex=get_path_play_index()
```

3.1.4.9 get_num_of_path

描述

获取轨迹的路径点个数。

参数

无。

返回值

int pointIndex: 轨迹索引，取值范围 0~49。

示例

```
num=get_num_of_path(5)           --获取索引值为 5 的轨迹中的路径点个数
```

3.1.5 外部轴功能函数

3.1.5.1 exj_config_ratio

描述

设定外部轴传动比。

参数

int exjRatio[]: 外部轴传动比，移动 1mm 对应的脉冲数(pulse/mm)或旋转 1° 对应的脉冲数(pulse/°)；

int exjNumber[]: 外部轴序号。

示例

```
(1) exj_config_ratio(500,0)           --设定外部轴 0 的传动比为 500 pulse/mm
(2) exj_config_ratio({1000,100},{0,1})
--设定外部轴 0 的传动比为 1000 pulse/° ， 外部轴 1 的传动比为 100 pulse/mm
```

3.1.5.2 exj_config_range

描述

设定外部轴范围限制。

参数

double exjMaxRange[]: 外部轴范围最大值(直线型单位 mm， 旋转型单位°)；

double exjMinRange[]: 外部轴范围最小值(直线型单位 mm， 旋转型单位°)；

int exjNumber[]: 外部轴序号。

示例

```
exj_config_range({300,1000},{-300,-1000},{0,1})
--设定外部轴 0 最大值 300° ， 最小值-300° ， 外部轴 1 最大值 1000mm， 最小值-1000mm
```

3.1.5.3 exj_config_zero

描述

设定外部轴零位。

参数

int exjZero[]: 外部轴零位对应的脉冲值；

int exjNumber[]: 外部轴序号。

示例

```
exj_config_zero({0,1000},{0,1})
--设定外部轴 0 的零位脉冲 0， 外部轴 1 的零位脉冲为 1000
```

3.1.5.4 exj_config_type

描述

设定外部轴类型。

参数

int exjType[]: 外部轴类型， 0-直线型 1-旋转型；

int exjNumber[]: 外部轴序号。

示例

```
exj_config_type(1,0)           --设定外部轴 0 为旋转型
exj_config_type({0,0},{1,2})  --设定外部轴 1 和 2 为直线型
```

3.1.5.5 exj_config_coordsys

描述

设定外部轴坐标系。

参数

double exjCoordsys[]: 外部轴坐标系在参考坐标系下的描述;

int exjNumber[]: 外部轴序号。

示例

```
exj_config_coordsys({{0,0,0,0,0,0},{0,0,100,0,0,90}},{0,1})
--设定外部轴 0 坐标系为{0,0,0,0,0,0}，外部轴 1 坐标系为{0,0,100,0,0,90}
```

3.1.5.6 exj_config_master

描述

设定外部轴主轴和主从比。

参数

int exjMaster[]: 外部轴主轴序号，-1 代表取消主轴;

double exjMasterRatio[]: 外部轴主从比(从轴运动的距离/主轴运动的距离);

int exjNumber[]: 外部轴序号。

示例

```
exj_config_master(0,1.0,1) --将外部轴 0 设定为外部轴 1 的主轴，主从比为 1.0
exj_config_master({0,0},{1.0,1.0},{1,2}) --将外部轴 0 设定为外部轴 1 和 2 的主轴，主从比为 1.0
exj_config_master({-1,-1},{1.0,1.0},{1,2}) --取消外部轴 1 和 2 的主轴设定
```

3.1.5.7 exj_enable

描述

外部轴使能。

参数

int exjNumber[]: 外部轴序号。

示例

```
(1) exj_enable(0)          --外部轴 0 使能
(2) exj_enable({1,2})     --外部轴 1 和 2 使能
```

3.1.5.8 exj_disable

描述

外部轴断使能。

参数

int exjNumber[]: 外部轴序号。

示例

```
(1) exj_disable(0)        --外部轴 0 断使能
(2) exj_disable({1,2})    --外部轴 1 和 2 断使能
```

3.1.5.9 exj_set_pos

描述

设定外部轴当前位置。

参数

double exjPos[]: 外部轴当前位置(直线型单位 mm，旋转型单位°)；

int exjNumber[]: 外部轴序号。

示例

```
exj_set_pos({0,100},{0,1})  --设定外部轴 0 当前位置为 0°，外部轴 1 当前位置为 100mm
```

3.1.5.10 exj_get_pos

描述

获取外部轴当前位置。

参数

int exjNumber[]: 外部轴序号。

返回值

double exjPos[]: 外部轴当前位置(直线型单位 mm，旋转型单位°)。

示例

```
exjPos = exj_get_pos({0,1})  --获取外部轴 0 和 1 当前位置，exjPos = {0,100}
```

3.1.5.11 exj_get_servo_pos

描述

获取外部轴伺服当前位置（32 位整数）。

参数

int exjNumber[]: 外部轴序号。

返回值

double exjPos[]: 外部轴当前位置(单位 pulse)。

示例

```
exjPos = exj_get_servo_pos(0)    --获取外部轴 0 伺服当前位置
```

3.1.5.12 exj_get_servo_pos64

描述

获取外部轴伺服当前位置（64 位整数），可记录 32 位位置溢出后的绝对位置。

参数

int exjNumber[]: 外部轴序号。

返回值

double exjPos[]: 外部轴当前位置(单位 pulse)

示例

```
exjPos = exj_get_servo_pos64(0)    --获取外部轴 0 伺服当前位置
```

3.1.5.13 exj_move

描述

外部轴运动到目标位置。

参数

double position[]: 目标位置(直线型单位 mm，旋转型单位°);

double velocity[]: 目标速度(直线型单位 mm/s，旋转型单位°/s);

double accTime: 加减速时间(单位为 ms);

int exjNumber[]: 外部轴序号。

示例

```
exj_config_type({0,0},{0,1})    --设定外部轴 0 和 1 为直线型
```

```
exj_move(300,100,100,0)
```

--外部轴 0 运动到 300mm 的位置，最大速度为 100mm/s，加减速时间为 100ms

```
exj_move({500,1000},{100,100},100,{0,1})
```

--外部轴 0 和外部轴 1 同步运动到 1000mm 和 500mm 的位置，最大速度都为 100mm/s，加减速时间为 100ms

3.1.5.14 exj_stop

描述

外部轴运动停止。

参数

double stopType: 停止类型：-1 为紧急停止；0 为按照当前加速度停止；大于 0 的数值为设定的减速距离(直线型单位 mm，旋转型单位°)；

int stopOnly: 是否直接停止，退出当前运动指令，0-否，1-是，选填参数；

int exjNumber: 外部轴序号。

示例

```
exj_stop(0,0)
```

--外部轴 0 按照当前的加速度停止

3.1.5.15 exj_continue

描述

单个轴继续运动，与 exj_stop 配合使用。

参数

int exjNumber: 外部轴序号。

示例

```
exj_stop(0)
```

--外部轴 0 按照当前的加速度停止

```
exj_continue(0)
```

--外部轴 0 继续运动

3.1.5.16 movel_exj

描述

机械臂与外部轴联动，直线运动至目标位置。

参数

double position[12]: 目标位置(数据格式 {x,y,z,rx,ry,rz,exj0,exj1,exj2,exj3,exj4,exj5}，其中 x, y, z, rx, ry,

rx 为机械臂 TCP 相对于目标坐标系的位置, x, y, z 单位为 mm, rx, ry, rz 以轴角的方式描述; exj0, exj1, exj2, exj3, exj4, exj5 分别为 0~5 号外部轴相对于零位的位置, 直线型单位 mm, 旋转型单位°;

double joint[6]: 参考关节角度(数据格式{j1,j2,j3,j4,j5,j6}, 单位为°), 选填参数;

double velocity[3]: 目标速度(数据格式{robotVelocity,exjVelocityL,exjVelocityR}, 其中 robotVelocity 为机械臂 TCP 速度, 单位为 mm/s; exjVelocityL 为外部轴直线速度, 单位为 mm/s; exjVelocityR 为外部轴旋转速度, 单位为°/s);

double acc: 加速度(单位为 mm/s²);

double blendRadius: 过渡半径(单位为 mm)。

示例

```
position={160,-550,300,180,0,0,100,1000,0,0,0,0}
--目标位置(机械臂 TCP 运动至 {160,-550,300,180,0,0}, 外部轴 0 运动至 100°, 外部轴 2 运动至 1000mm)
velocity={500,500,50}
--目标速度(TCP 速度 500mm/s, 外部轴直线速度 500mm/s, 外部轴旋转速度 50°/s)
movej_exj(position,velocity,1000,0)
--机械臂和外部轴以目标速度直线联动至目标位置, 加速度 1000mm/s2, 过渡半径 0mm
```

3.1.5.17 movej_exj

描述

机械臂与外部轴联动, 关节运动至目标位置。

参数

double position[12]: 目标位置(数据格式{j1,j2,j3,j4,j5,j6,exj0,exj1,exj2,exj3,exj4,exj5}, 其中 j1, j2, j3, j4, j5, j6 为机械臂关节目标位置, 单位为°; exj0, exj1, exj2, exj3, exj4, exj5 分别为 0~5 号外部轴相对于零位的位置, 直线型单位 mm, 旋转型单位°);

double velocity[3]: 目标速度(数据格式{jointVelocity,exjVelocityL,exjVelocityR}, 其中 jointVelocity 为机械臂关节速度, 单位为°/s; exjVelocityL 为外部轴直线速度, 单位为 mm/s; exjVelocityR 为外部轴旋转速度, 单位为°/s);

double acc: 加速度(单位为°/s²);

double blendRadius: 过渡半径(单位为 mm)。

示例

```
position={0,0,-90,0,90,0,100,0,0,0,0,0}

--目标位置(机械臂关节运动至{0,0,-90,0,90,0}，外部轴 0 运动至 100° )

velocity={50,500,50}

--目标速度(关节速度 50° /s，外部轴直线速度 500mm/s，外部轴旋转速度 50° /s)

movej_exj(position,velocity,80,0)

--机械臂和外部轴以目标速度关节运动至目标位置，加速度 80° /s2，过渡半径 0mm
```

3.1.5.18 movejp_exj

描述

机械臂与外部轴联动，关节运动至目标位置。

参数

double position[12]: 目标位置(数据格式{x,y,z,rx,ry,rz,exj0,exj1,exj2,exj3,exj4,exj5}, 其中 x, y, z, rx, ry, rz 为机械臂 TCP 相对于目标坐标系的位置, x, y, z 单位为 mm, rx, ry, rz 以轴角的方式描述; exj0, exj1, exj2, exj3, exj4, exj5 分别为 0~5 号外部轴相对于零位的位置, 直线型单位 mm, 旋转型单位° ;

double joint[6]: 参考关节角度(数据格式{j1,j2,j3,j4,j5,j6}, 单位°), 选填参数;

double velocity[3]: 目标速度(数据格式{jointVelocity,exjVelocityL,exjVelocityR}, 其中 jointVelocity 为机械臂关节速度, 单位为° /s; exjVelocityL 为外部轴直线速度, 单位为 mm/s; exjVelocityR 为外部轴旋转速度, 单位为° /s);

double acc: 加速度(单位为° /s2);

double blendRadius: 过渡半径(单位为 mm)。

示例

```
position={160,-550,300,180,0,0,100,0,0,0,0,0}

--目标位置(机械臂关节运动至 TCP 到达{160,-550,300,180,0,0}，外部轴 0 运动至 100° )

velocity={50,500,50}

--目标速度(关节速度 50° /s，外部轴直线速度 500mm/s，外部轴旋转速度 50° /s)

movejp_exj(position,velocity,80,0)

--机械臂和外部轴以目标速度关节运动至目标位置，加速度 80° /s2，过渡半径 0mm
```

3.1.5.19 movec_exj

描述

机械臂与导轨联动，圆弧运动至目标位置。

参数

double position1[12]: 目标位置(数据格式{x,y,z,rx,ry,rz,exj0,exj1,exj2,exj3,exj4,exj5}, 其中 x, y, z, rx, ry, rz 为机械臂 TCP 相对于目标坐标系的位置, x, y, z 单位为 mm, rx, ry, rz 以轴角的方式描述; exj0, exj1, exj2, exj3, exj4, exj5 分别为 0~5 号外部轴相对于零位的位置, 直线型单位 mm, 旋转型单位° ;

double joint1[6]: 参考关节角度 1(数据格式{j1,j2,j3,j4,j5,j6}, 单位°), 选填参数;

double position2[12]: 目标位置(数据格式{x,y,z,rx,ry,rz,exj0,exj1,exj2,exj3,exj4,exj5}, 其中 x, y, z, rx, ry, rz 为机械臂 TCP 相对于目标坐标系的位置, x, y, z 单位为 mm, rx, ry, rz 以轴角的方式描述; exj0, exj1, exj2, exj3, exj4, exj5 分别为 0~5 号外部轴相对零位的位置, 直线型单位 mm, 旋转型单位° ;

double joint2[6]: 参考关节角度 2(数据格式{j1,j2,j3,j4,j5,j6}, 单位°), 选填参数;

double velocity[3]: 目标速度(数据格式{robotVelocity,exjVelocityL,exjVelocityR}, 其中 robotVelocity 为机械臂 TCP 速度, 单位为 mm/s; exjVelocityL 为外部轴直线速度, 单位为 mm/s; exjVelocityR 为外部轴旋转速度, 单位为° /s);

double acc: 加速度(单位为 mm/s²);

double blendRadius: 过渡半径(单位为 mm);

int fixedRot: 姿态方向模式, 0-由起点姿态变换到终点姿态 1-姿态绕圆轴线旋转, 选填参数;

double specAngle: 圆弧角, 圆弧弧长对应的角度, 非零时生效, 选填参数。

示例

```
position1={160,-550,300,180,0,0,100,1000,0,0,0,0}
--目标位置 1(机械臂 TCP 运动至{160,-550,300,180,0,0}, 外部轴 0 运动至 100°, 外部轴 2 运动至 1000mm)

position2={300,-400,300,180,0,0,100,1000,0,0,0,0}
--目标位置 2(机械臂 TCP 运动至{300,-400,300,180,0,0}, 外部轴 0 运动至 100°, 外部轴 2 运动至 1000mm)

velocity={500,500,50}
--目标速度(TCP 速度 500mm/s, 外部轴直线速度 500mm/s, 外部轴旋转速度 50° /s)
```

```
movec_exj(position1,position2,velocity,1000,0,0,0)
```

--机械臂和外部轴以目标速度圆弧联动至目标位置，加速度 1000mm/s²，过渡半径 0mm

3.1.6 轨迹叠加功能函数

3.1.6.1 start_path_offset

描述

开启轨迹偏移。

参数

int type: 轨迹偏移参考坐标系类型，0-关节坐标系、1-基坐标系、2-TCP 坐标系、3-用户坐标系、4-轨迹坐标系；

double coordsys[6]: 参考坐标系，注：可省略；

int realDH: 是否使用真实 DH 参数，注：可省略，默认为使用。

示例

(1) start_path_offset(1,0) --开启轨迹偏移，参考坐标系为基坐标系，不使用真实 DH 参数

(2) start_path_offset(3,{100,200,100,0,0,90}) --开启轨迹偏移，参考坐标系为{100,200,100,0,0,90}

3.1.6.2 end_path_offset

描述

结束轨迹偏移。

参数

无。

示例

```
end_path_offset() --结束轨迹偏移
```

3.1.6.3 set_path_offset

描述

设定目标轨迹偏移量。

参数

double offset[6]: 目标轨迹偏移量。

示例

```
set_path_offset({0,0,10,0,0,0})    --设定目标轨迹偏移量
```

3.1.6.4 get_path_offset

描述

获取当前轨迹偏移量。

参数

double offset[6]: 当前轨迹偏移量。

示例

```
offset = {0,0,0,0,0,0}
offset = get_path_offset()    --获取当前轨迹偏移量
```

3.1.6.5 set_path_offset_limit

描述

设定轨迹偏移量限制。

参数

double limits[6]: 轨迹偏移量限制。

示例

```
set_path_offset_limit({300,300,300,50,50,50})    --设定轨迹偏移量限制为{300,300,300,50,50,50}
```

3.1.6.6 set_path_offset_filter

描述

设定轨迹偏移滤波系数。

参数

double filter: 滤波系数，取值范围 0~1，值越接近 1，轨迹偏移响应越快。

示例

```
set_path_offset_filter(0.2)    --设定轨迹偏移滤波系数为 0.2
```

3.1.6.7 set_path_offset_speed

描述

设定轨迹偏移速度限制。

参数

double speed[6]: 轨迹偏移速度限制。

示例

```
start_path_offset({50,50,50,20,20,20})    --设定轨迹偏移速度限制
```

3.2 内部函数

3.2.1 时间函数

3.2.1.1 wait

描述

程序等待一段时间，输入参数为时间长度，单位为 ms。

参数

double time。

示例

```
wait(1000)    --等待 1000ms
```

3.2.1.2 get_time

描述

获取当前的世界时间。

参数

无。

返回值

{年,月,日,时,分,秒,毫秒}。

示例

```
a=get_time()    --获取世界时间赋值到 a，得到数组{年，月，日，小时，分，秒，毫秒}
```

3.2.1.3 get_sys_time

描述

获取控制器上电后的时间，可以以此来做时间戳记录。

参数

无。

返回值

{秒，微秒}。

示例

```
a=get_systime() --获取控制器上电时间赋值到 a，得到数组{秒，微秒}
```

3.2.2 程序控制

3.2.2.1 task_create

描述

任务创建。

参数

coroutine task。

示例

```
function CGXIProgram()
    while(true)
    do
        movej({10,10,10,10,10,10},60,80,0)
        movej({20,20,20,20,20,20},60,80,0)
        wait(1)
    end
end
robotProgramResult=task_create(CGXIProgram)
```

3.2.2.2 task_pause

描述

任务暂停。

参数

coroutine task。

示例

```
function CGXIProgram()
    while(true)
    do
```

```
        movej({10,10,10,10,10,10},60,80,0)

        movej({20,20,20,20,20,20},60,80,0)

        wait(1)

    end

end

robotProgramResult=task_create(CGXIProgram)

wait(2000)

task_pause(robotProgramResult)
```

3.2.2.3 task_continue

描述

任务继续，必须在机器人回到暂停位置后才可以执行，否则报错。

参数

coroutine task。

示例

```
function CGXIProgram()

    while(true)

    do

        movej({10,10,10,10,10,10},60,80,0)

        movej({20,20,20,20,20,20},60,80,0)

        wait(1)

    end

end

robotProgramResult=task_create(CGXIProgram)           --创建任务

function Task_1()

    while(true)

    do

        wait(1000)

        task_pause(robotProgramResult);                --暂停任务

    end

end
```

```

        wait(1000)

        tarJoint=get_move_target_joint();           --记录停止时目标位置

        waypoint_5_q={0,0,0,0,0,0}

        movej(waypoint_5_q,60,80,0)

        waypoint_6_q={20,20,0,0,0,0}

        movej(waypoint_6_q,60,80,0)

        wait(1000);

        movej(tarJoint,60,80,0)                     --恢复停止时的目标位置

        wait(1000);

        task_continue(robotProgramResult);          --继续任务

        wait(1)  --sync

    end

end

Task_1Result=task_create(Task_1)

```

3.2.2.4 task_delete

描述

任务停止，并且删除此任务，如需重新执行，需要执行 task_create。

参数

coroutine task。

示例

```

function CGXIProgram()

    while(true)

    do

        movej({10,10,10,10,10,10},60,80,0)

        movej({20,20,20,20,20,20},60,80,0)

        wait(1)

    end

end

end

```

```
robotProgramResult=task_create(CGXIProgram)

task_delete(robotProgramResult)
```

3.2.2.5 halt

描述

程序停止，运动停止。

参数

无。

示例

```
halt()                                --停止
```

3.2.2.6 pause

描述

程序暂停。

参数

无。

示例

```
pause()                               --暂停
```

3.2.2.7 event_task_create

描述

创建事件。

参数

int irqcode 触发事件编码(详见下表);

int irqtype 触发类型编码(详见下表);

int usrdata 用户数据(保留);

function irqhandler 事件处理函数。

示例

```
ditrigcount0=0

function DI0_IRQ()

    while(true)
```

```

do

    ditrigcount0 = ditrigcount0 +1;

    print(ditrigcount0)

    event_suspend()                --挂起本次事件

    wait(1)                        --sync

end

end

DI0_IRQResult=event_task_create(1,1,0,DI0_IRQ)
    
```

注：事件处理函数中程序应尽量简短，快速处理完成，不建议程序中存在 wait 和 move 等指令。注册一个基于 DI0 上升沿事件的处理函数 DI0_IRQ，当上升沿事件触发后，打印触发次数。触发事件编码见表 7。

表 7 触发事件编码

触发源	事件编码	说明
EVENT_DI0	1	标准数字量输入 0
EVENT_DI1	2	标准数字量输入 1
EVENT_DI2	3	标准数字量输入 2
EVENT_DI3	4	标准数字量输入 3
EVENT_EQEP0_STROBE	20	编码器 Strobe 选通信号 0
EVENT_EQEP1_STROBE	21	编码器 Strobe 选通信号 1

触发类型编码见表 8。

表 8 触发类型编码

触发类型	类型编码	说明
EVENT_TRIG_RISE_EDGE	1	数字 IO 口上升沿触发
EVENT_TRIG_FALL_EDGE	2	数字 IO 口下降沿触发
EVENT_TRIG_EQEP_STROBE	5	编码器 Strobe 信号触发

3.2.2.8 event_suspend

描述

一般放在用户函数末尾，用户事件函数为一次性执行模式；若无此函数，用户事件函数进入循环执行模式。

参数

无。

示例

```
ditrigcount0=0

function DI0_IRQ()

    while(true)

        do

            ditrigcount0 = ditrigcount0 +1;

            print(ditrigcount0)

            event_suspend()                --挂起本次事件

            wait(1)                        --sync

        end

    end

end

DI0_IRQResult=event_task_create(1,1,0,DI0_IRQ)
```

3.2.2.9 rt_event_register

描述

注册指定编码的实时事件，对实时性要求不高的应用建议优先采用用户注册函数 `event_task_create`。

注：事件处理函数中程序应尽量简短，快速处理完成。

参数

int eventcode: 事件编码 (1 - 3);

function irqhandler: 事件处理函数，函数无参数、无返回值。

示例

```
var_1=0

function rt_event_func()

    var_1 = var_1 +1;

end

rt_event_register(1, rt_event_func)

--注册一个编码为 1 的实时事件函数 rt_event_func，周期性触发执行
```

3.2.2.10 rt_event_unregister

描述

注销指定编码的实时事件。

参数

int eventcode: 触发事件编码 (1 - 3)。

示例

```
rt_event_unregister(1)           --注销编码为 1 的实时事件。
```

3.2.2.11 getlines

描述

获取 lua 当前所有用户任务的运行行号。

参数

无。

返回值

table linesTable: 行号。

示例

```
linesTable = getlines()

for index, value in ipairs(linesTable) do
    print(index, value)
end

wait(100)
```

3.2.3 机器人系统函数

3.2.3.1 set_tcp

描述

设置工具末端相对于法兰的偏移向量。

参数

double tcpOffset[6]: 工具末端相对于工具法兰的偏移向量。

示例

```
set_tcp{0,0,-10,0,0,0}          --工具末端相对于法兰沿 z 轴负方向偏移 10cm
```

3.2.3.2 set_tcp_payload

描述

设置负载质量和重心。

参数

double mass:负载质量;

double center[3]:质心坐标（法兰坐标系下表达 mm）。

示例

```
set_tcp_payload(5,{0,0,45})
```

 --负载质量 5kg，质心为法兰 z 轴正向 45mm

3.2.3.3 get_tcp

描述

获取工具末端相对于法兰的偏移向量。

参数

无。

返回值

double tcpOffset[6]:工具末端相对于工具法兰的偏移向量。

示例

```
a={0,0,0,0,0,0}
a = get_tcp()
```

 --获取工具末端相对于工具法兰的偏移向量并赋值到 a

3.2.3.4 get_target_tcp_pose

描述

获取当前工具末端相对于基坐标系的目标位姿(tcp 位姿)，为瞬时目标 tcp 位姿。

参数

无。

返回值

double tcpPose[6]: 当前工具末端相对于基坐标系的目标位姿。

示例

```
a={0,0,0,0,0,0}
a=get_target_tcp_pose()
```

 --获取当前工具末端相对于基坐标系的目标位姿并赋值给 a

注：与 get_move_target_tcp_pose()函数的区别为：get_move_target_tcp_pose()的返回值为当前运动指令最终的目标位置，get_target_tcp_pose()的返回值为当前瞬间控制器生成的目标 tcp 位置。

3.2.3.5 get_target_tcp_offset

描述

获取机械臂目标 TCP。

参数

无。

返回值

double[6] tcp: 工具末端相对于基坐标系的偏移向量。

示例

```
a={0,0,0,0,0,0}
a=get_target_tcp_offset()           --获取机械臂目标 TCP 并赋值给 a
```

3.2.3.6 get_move_target_tcp_pose

描述

获取当前运动的最终目标 TCP 位姿。

参数

无。

返回值

double moveTcpPose[6]: 当前运动的最终目标 TCP 位姿。

示例

```
a={0,0,0,0,0,0}
a=get_move_target_tcp_pose()        --获取当前运动的最终目标 TCP 位姿并赋值给 a
```

3.2.3.7 get_move_target_joint

描述

获取当前运动的实时目标关节角度。

参数

无。

返回值

double targetJoint[6]: 当前运动的实时目标关节角度。

示例

```
a={0,0,0,0,0,0}
```

```
a=get_move_target_joint()
```

--获取当前运动的实时目标关节角度并赋值给 a

3.2.3.8 get_tcp_force(coordinate)

描述

获取当前工具端力/力矩矢量，输入参数 coordinate 代表坐标系的选择(0 为基坐标系，1 为工具坐标系)。

示例

```
a={0,0,0,0,0,0}
```

```
a=get_tcp_force(0)
```

--获取当前工具端力/力矩矢量对应 x,y,z,Rx,Ry,Rz

3.2.3.9 get_joint_torque_external()

描述

获取当前各个关节的力矩。

示例

```
a={0,0,0,0,0,0}
```

```
a=get_joint_torque_external()
```

--获取当前各个关节的力矩

3.2.3.10 get_actual_tcp_pose

描述

获取当前工具末端相对于基坐标系得实际位姿（TCP 位姿），为瞬时实际 TCP 位姿。

参数

无。

返回值

double TcpPose[6]: 瞬时实际 TCP 位姿。

示例

```
a={0,0,0,0,0,0}
```

```
a=get_actual_tcp_pose()
```

3.2.3.11 get_actual_tcp_speed

描述

获取当前运动的实际 TCP 速度矢量。

参数

无。

返回值

double TcpSpeed[6]: 当前运动的实际 TCP 速度矢量, 分别为 x,y,z,rx,ry,rz 方向的速度, 其中 rx, ry, rz 为轴角姿态描述方式。

示例

```
a={0,0,0,0,0,0}
a=get_actual_tcp_speed()
```

3.2.3.12 get_actual_joint_positions**描述**

获取当前运动的实时关节角度。

参数

无。

返回值

double ActualJoint[6]: 当前运动的实时关节角度。

示例

```
a={0,0,0,0,0,0}
a=get_actual_joint_positions()
```

3.2.3.13 get_actual_joint_speeds**描述**

获取当前运动的实时关节速度。

参数

无。

返回值

double JointSpeed[6]: 当前运动的实际关节速度。

示例

```
a={0,0,0,0,0,0}
a=get_actual_joint_speeds()
```

3.2.3.14 get_actual_tcp_force

描述

获取当前工具端实际力/力矩矢量。

参数

int coordinate: 选择的坐标系(0 为基坐标系, 1 为工具坐标系)。

返回值

double TcpForce[6]: 当前工具端实际力/力矩矢量。

示例

```
a={0,0,0,0,0,0}
a=get_actual_tcp_force(0)
```

3.2.3.15 set_percentvelocity

描述

设置并保存机器人速度百分比。

参数

int percentvelocity 范围 1 - 100。

返回值

无。

示例

```
set_percentvelocity(80)
```

3.2.3.16 get_percentvelocity

描述

获取当前机器人速度百分比。

参数

无。

返回值

int percentvelocity。

示例

```
pv = get_percentvelocity()
```

3.2.3.17 sync

描述

同步。

参数

无。

返回值

无。

示例

```
sync()
```

3.2.3.18 get_actual_joint_torques

描述

获取机械臂实际关节转矩。

参数

无。

返回值

double[6] tor: 机械臂实际关节转矩。

示例

```
a={0,0,0,0,0,0}
```

```
a=get_actual_joint_torques()
```

3.2.3.19 get_target_joint_torques

描述

获取机械臂目标关节转矩。

参数

无。

返回值

double[6] tor: 机械臂目标关节转矩。

示例

```
a={0,0,0,0,0,0}
```

```
a=get_target_joint_torques()
```

3.2.3.20 set_torprot_plus

描述

用于修改转矩持续超限报警的转矩阈值，通过浮点 D21048 监测。

参数

double tor_plus: 转矩持续超限报警的转矩阈值相对于默认值的变化量，转矩阈值默认为 800。

返回值

无。

示例

```
set_torprot_plus(10)                --转矩持续超限报警的转矩阈值改为 810
```

3.2.3.21 set_mechanical_unit

描述

设定机械单元。

参数

int unitId: 机械单元序号，1~10；

int exjNumber: 机械单元包含的外部轴数量，1~6；

int exjId[]: 机械单元包含的外部轴对应的序号；

int isFixed: 是否固定不变，0-移动类型，1-固定类型；

double coordSys[6]: 基坐标系，若机械臂单元为固定类型表示相对于世界坐标系的位姿，若机械单元为移动类型则表示相对于被绑定机械单元最末端轴坐标系的位姿；

int unit: 基坐标系绑定的机械单元，移动类型时需要设定，固定类型时无效。

示例

```
set_mechanical_unit(1,2,{0,1},1,{0,0,0,0,0,0},0)
--设定机械单元 1，包含 0 和 1 两个外部轴，该机械单元为固定类型，基坐标系相对于世界坐标系
为{0,0,0,0,0,0}
```

3.2.3.22 set_wcs

描述

设定工件坐标系。

参数

double coordSys[6]: 工件坐标系在参考坐标系下的描述，固定类型在世界坐标系下定义，移动类型在设定的机械单元坐标系下定义（数据格式{x,y,z,rx,ry,rz}, x, y, z 单位为 mm, rx, ry, rz 以轴角的方式描述）;

int isHandheld: 工件坐标系是否固定在机械臂末端，0-外部工件，1-手持工件；

int isFixed: 工件坐标系是否固定不变，0-移动类型，1-固定类型；

int bindUnit: 参考坐标系绑定的机械单元，移动类型时需要设定，固定类型时无效。

示例

```
set_wcs({100,0,0,0,0,0},0,1,0)
```

--设定机械臂外部的固定工件坐标系，工件坐标系在世界坐标系下的描述为{100,0,0,0,0,0}

3.3 计算函数

3.3.1 三角函数

3.3.1.1 sin

描述

输入参数为角度，单位°，返回值为正弦值。

参数

double angle。

返回值

double value。

示例

```
a = sin(30)
```

3.3.1.2 cos

描述

输入参数为角度，单位°，返回值为余弦值。

参数

double angle。

返回值

double value。

示例

```
a = cos(60)
```

3.3.1.3 tan

描述

输入参数为角度，单位°，返回值为正切值。

参数

double angle。

返回值

double value。

示例

```
a = tan(45)
```

3.3.1.4 asin

描述

输入参数为正弦值，返回值为角度，单位°，范围[-180°，180°]。

参数

double value。

返回值

double angle。

示例

```
a = asin(0.5)
```

3.3.1.5 acos

描述

输入参数为余弦值，返回值为角度，单位°，范围[-180°，180°]。

参数

double value。

返回值

double angle。

示例

```
a = acos(0.5)
```

3.3.1.6 atan

描述

输入参数为正切值，返回值为角度，单位°，范围[-180°，180°]。

参数

double value。

返回值

double angle。

示例

```
a = atan(1)
```

3.3.1.7 atan2

描述

输入参数为 x, y 的坐标轴，返回值为角度，单位°，范围[-180°，180°]。

参数

double x;

double y。

返回值

double angle。

示例

```
a = atan(10,10)
```

3.3.2 位姿函数

3.3.2.1 pose_add

描述

{x,y,z}相加，姿态结果为两个姿态右乘叠加的结果，在此位置和姿态独立处理，与 postrans 并不一样。

参数

double pose_from[6];

double pose_increment[6]。

返回值

double pose_to[6]。

示例

```
waypoint_1_p={-259.50,608.17,388.05,-118.11,-46.09,-20.26}    --点 1 姿态
waypoint_2_p={23,245,-230,333,44,56}                        --点 2 姿态
a=pose_add(waypoint_1_p,waypoint_2_p)                        --点 1 与点 2 姿态右乘叠加
```

3.3.2.2 pose_sub

描述

{x,y,z}相减，姿态结果为 pose_from 右乘 pose_decrement 逆叠加的结果，在此位置和姿态独立处理，与 postrans 并不一样。

参数

double pose_from[6];

double pose_decrement[6]。

返回值

double pose_to[6]。

示例

```
waypoint_1_p={-259.50,608.17,388.05,-118.11,-46.09,-20.26}    --点 1 姿态
waypoint_2_p={23,245,-230,333,44,56}                        --点 2 姿态
a=pose_sub(waypoint_1_p,waypoint_2_p)
```

3.3.2.3 pose_trans

描述

位姿整体右乘叠加计算。

参数

double pose_from[6];

double pose_trans[6]。

返回值

double pose_to[6]。

示例

```

x_int=114.3
x_dec=10
y_int=-492
y_dec=10
angle_int=-180
angle_dec=10
x_offset=x_int+0.001*x_dec
y_offset=y_int+0.001*y_dec
angle_offset=angle_int+0.001*angle_dec
--将 x_offset、y_offset、angle_offset 进行运算
poseToffset={x_offset,y_offset,0,0,0,0}
poseRoffset={0,0,0,0,0,angle_offset}
pose=pose_trans(poseToffset,poseRoffset)

```

3.3.2.4 point_dist

描述

两个路点位置距离计算。

参数

double pose1[6];

double pose2[6]。

返回值

return double distance。

示例

```

waypoint_1_p={-259.50,608.17,388.05,-118.11,-46.09,-20.26}    --点 1 位姿
waypoint_2_p={23,245,-230,333,44,56}                        --点 2 位姿
a=point_dist(waypoint_1_p,waypoint_2_p)                    --计算两个路点位置距离

```

3.3.2.5 pose_inv

描述

位姿整体逆计算。

参数

double pose[6]。

返回值

return double pose_inv[6]。

示例

```
waypoint_1_p={-259.50,608.17,388.05,-118.11,-46.09,-20.26}    --点 1 位姿
a=pose_inv(waypoint_1_p)    --位姿整体逆计算
```

3.3.2.6 pose_offset

描述

位姿沿指定坐标系偏移。

参数

double pose[6]: 基坐标系下工具位姿（数据格式 {x,y,z,rx,ry,rz}, x, y, z 为位置，单位为 mm，rx, ry, rz 为姿态，以轴角的方式描述，单位为°）；

int type: 偏移参考坐标系，0-基坐标系、1-工具坐标系、2-用户坐标系；

double offset[6]: 参考坐标系下偏移量（数据格式 {x,y,z,rx,ry,rz}, x, y, z 为位置偏移量，单位为 mm，rx, ry, rz 为姿态偏移量，以轴角的方式描述，单位为°）；

double coordsys[6]: 用户坐标系，选填参数，参考坐标系为用户坐标系时必须填写，其他情况下可不填写（数据格式 {x,y,z,rx,ry,rz}, x, y, z 为位置，单位为 mm，rx, ry, rz 为姿态，以轴角的方式描述，单位为°）。

返回值

double poseOffset[6]: 偏移后的位姿（数据格式 {x,y,z,rx,ry,rz}, x, y, z 为位置，单位为 mm，rx, ry, rz 为姿态，以轴角的方式描述，单位为°）。

示例 1

```
pose = {500, -100, 400, -127.28, -127.28, 0}
type = 0
offset = {100, 0, 0, 0, 0, 0}
poseOffset = pose_offset(pose, type, offset)
--位姿沿基坐标系偏移，偏移后位姿为{600,-100,400,127.278,127.278,0}
```

```
type = 1

offset = {0, 0, 50, 0, 0, 30}

poseOffset = pose_offset(pose, type, offset)

--位姿沿工具坐标系偏移，偏移后位姿为{500,-100,350,155.884,90,0}

type = 2

offset = {0, 60, 0, 0, 50, 30}

coordsys = {100, 200, 200, 50, 90, 100}

poseOffset = pose_offset(pose, type, offset, coordsys)

--位姿沿用户坐标系偏移，偏移后位姿为{625.018,299.883,303.839,-49.444,-147.021,-9.98}
```

示例 2

```
--移动到起始点

home = {300, -100, 400, -127.28, -127.28, 0}

movejp(home, {0, 0, -90, 0, 90, 0}, 60, 80, 0)

--位姿沿基坐标系偏移，获取打孔点

pose = pose_offset(home, 0, {100, 0, -100, 0, 0, 0})

--3X3 阵列打孔点

for i = 0 , 2 do

    for j = 0 , 2 do

        offset = {i*50, j*50, 0, 0, 0, 0}

        coordSys = {300, -100, 400, 0, 0, 0}

        --位姿沿用户坐标系偏移，计算第 N 个打孔点

        poseN = pose_offset(pose, 2, offset, coordSys)

        --位姿沿工具坐标系偏移，移动到打孔过渡点

        movel(pose_offset(poseN, 1, {0, 0, -50, 0, 0, 0}), 250, 600, 20)

        --移动到打孔点

        movel(poseN, 250, 600, 0)

        --位姿沿工具坐标系偏移，移动到打孔过渡点

        movel(pose_offset(poseN, 1, {0, 0, -50, 0, 0, 0}), 250, 600, 20)
```

```
end  
  
end
```

3.3.2.7 euler2rotvec

描述

欧拉角转换为轴角，单位为°。

参数

double euler[3]。

返回值

return double rotvec[3]。

示例

```
euler={30,60,90}  
  
rotvec=euler2rotvec(euler)
```

3.3.2.8 rotvec2euler

描述

轴角转换为欧拉角，单位为°。

参数

double rotvec[3]。

返回值

return double euler[3]。

示例

```
rotvec={30,60,90}  
  
euler=rotvec2euler(rotvec)
```

3.3.2.9 inverse_kine_judge

描述

根据法兰中心点坐标位置计算逆解是否存在。

参数

double pose[6]。

返回值

bool result。

示例

```
waypoint_1_p={-259.50,608.17,388.05,-118.11,-46.09,-20.26}
```

```
a=inverse_kine_judge(waypoint_1_p)
```

--若返回值为 true，则逆解存在，若为 false，则逆解不存在

3.3.2.10 inverse_kine

描述

根据法兰中心点坐标位置计算关节角度。

参数

double pose[6]。

返回值

double jointpose[6]。

示例

```
waypoint_1_p={-259.50,608.17,388.05,-118.11,-46.09,-20.26}    --法兰中心点坐标位置
```

```
--waypoint_1_q={-75.27,24.37,94.27,-83.18,-99.84,147.40}
```

```
a=inverse_kine(waypoint_1_p)    --根据法兰中心点坐标位置计算关节角度
```

3.3.2.11 forward_kine

描述

根据关节角度计算法兰中心点坐标。

参数

double jointpose[6]。

返回值

double pose[6]。

示例

```
waypoint_1_q={-75.27,24.37,94.27,-83.18,-99.84,147.40}    --关节角度
```

```
--waypoint_1_p={-259.50,608.17,388.05,-118.11,-46.09,-20.26}
```

```
a=forward_kine(waypoint_1_q)    --根据关节角度计算法兰中心点坐标
```

3.3.2.12 binary_list_to_integer

描述

bool 序列转整数。

参数

bool binary_list。

返回值

int integer。

示例

```
a=binary_list_to_integer{0,0,1,1,0,0,1}
```

--bool 序列转整数，低位在前，高位在后，原数据为 01001100

3.3.2.13 integer_to_binary_list

描述

整数转 bool 序列。

参数

int integer。

返回值

bool binary_list。

示例

```
a=integer_to_binary_list(76)
```

--整数转 bool 序列，低位在前高位在后，结果为 0011001，整理后为 01001100

3.3.2.14 linear_interp_pose

描述

根据起始位置和目标位置，以及插值比例，计算按照该比例线性插值的位置和姿态，插值比例范围为 0~1，当插值比例为 0 时，返回值与起始点一致，当插值比例为 1 时，返回终点位姿。

参数

double startPose[6]: 起始点位姿；

double endPose[6]: 终点位姿；

double ratio: 插值比例，范围为（0~1）。

返回值

double interpPose[6]; 插值点位姿。

示例

```
w_1_p={114.3,-492,490.9,-180,-1.3411e-6,8.15762e-6}
w_2_p={322.665,-405.526,490.898,-180,-1.3411e-6,1.01251e-5}
w_3_p=linear_interp_pose(w_1_p,w_2_p,0.5)
```

3.3.3 数值运算函数

3.3.3.1 swap_bytes_for_word

描述

双字节整数逆转字节序。

参数

int data1 双字节整数。

返回值

int data2 数据 1 字节逆转后的值。

示例

```
ret_data = swap_bytes_for_word(0x0102)    --双字节整数逆转字节序
```

3.3.3.2 swap_bytes_for_dword

描述

四字节整数/浮点逆转字节序。

参数

int data1 四字节整数或者 float data1 四字节浮点。

返回值

数据 1 字节逆转后的值。

示例 1

```
ret_data = swap_bytes_for_dword(0x01020304)    --四字节整数逆转字节序
```

示例 2

```
ret_data = swap_bytes_for_dword(30.0)    --浮点逆转字节序
```

3.3.3.3 swap_words_for_dword

描述

四字节整数/浮点逆转字序。

参数

int data1 四字节整数或者 float data1 四字节浮点。

返回值

数据 1 按字逆转后的值。

示例 1

```
ret_data = swap_words_for_dword(0x01020304)    --四字节整数逆转字序
```

示例 2

```
ret_data = swap_words_for_dword(30.0)          --四字节浮点逆转字序
```

3.3.3.4 commuvar_to_string

描述

通讯变量转换成字符串。

参数

int num 要转换的字节数；

CommuVarType regType 通讯变量类型 0 为 bool 变量 1 为 int16 2 为 int32 3 为 float；

int destRegIndex 通讯变量起始偏移量。

示例

```
str = commuvar_to_string(3,1,0)    --通讯变量类型为 int16，转换字节数为 3，起始偏移量为 0
```

3.3.3.5 string_to_commuvar

描述

字符串转换成通讯变量。

参数

CommuVarType regType 通讯变量类型：0 为 bool 变量，1 为 int16，2 为 int32，3 为 float；

int destRegIndex 通讯变量起始偏移量；

const char *str 待转换字符串。

示例

```
string_to_commuvar(1,0,"(9,2)")
```

--通讯变量类型为 int16，起始偏移量为 0，待转换字符串为"(9,2)"

3.3.4 位运算函数

3.3.4.1 bit_not

描述

按二进制位进行"取反"运算。

参数

int value: 待计算值。

返回值

int res: value 按二进制"取反"后的值。

示例

```
res=bit_not(5)                --res=-6
```

3.3.4.2 bit_and

描述

按二进制位进行"与"运算。

参数

int value1: 待计算值 1；

int value2: 待计算值 2。

返回值

int res: value1 和 value2 按二进制"与"运算后的值。

示例

```
res=bit_and(6,10)             --res=2
```

3.3.4.3 bit_or

描述

按二进制位进行"或"运算。

参数

int value1: 待计算值 1；

int value2: 待计算值 2。

返回值

int res: value1 和 value2 按二进制"或"运算后的值。

示例

```
res=bit_or(6,10)          --res=14
```

3.3.4.4 bit_xor

描述

按二进制位进行"异或"运算。

参数

int value1: 待计算值 1;

int value2: 待计算值 2。

返回值

int res: value1 和 value2 按二进制"异或"运算后的值。

示例

```
res=bit_xor(6,10)         --res=12
```

3.3.4.5 bit_lshift

描述

按二进制位进行"左移"运算。

参数

int value: 待计算值;

int offset: 左移的位数。

返回值

int res: value 按二进制"左移"offset 位后的值。

示例

```
res=bit_lshift(6,2)       --res=24
```

3.3.4.6 bit_rshift

描述

按二进制位进行"右移"运算。

参数

int value: 待计算值;

int offset: 右移的位数。

返回值

int res: value 按二进制"右移"offset 位后的值。

示例

```
res=bit_rshift(6,2)          --res=1
```

3.3.5 其他计算函数

3.3.5.1 ceil

描述

向上取整计算。

参数

double value: 待取整值

返回值

double res value: 向上取整的值。

示例

```
res=ceil(9.1)                --res=10
```

3.3.5.2 floor

描述

向下取整计算。

参数

double value: 待取整值

返回值

double res value: 向下取整的值。

示例

```
res=floor(9.9)                --res=9
```

3.3.5.3 random

描述

取随机数。

参数

无。

返回值

double res 0~1 之间的随机数。

示例

```
res=random()
```

--res 为 0~1 之间的随机数

3.3.5.4 log

描述

对数计算。

参数

double x: 底数;

double n: 真数。

返回值

double y: 以 x 为底 n 的对数。

示例

```
y=log(2,8)
```

--y=3

3.3.5.5 pow

描述

幂计算。

参数

double x: 基数;

double y: 指数。

返回值

double n: x 的 y 次幂。

示例

```
n=pow(2,3)
```

--n=8

3.3.5.6 sqrt

描述

开平方。

参数

double value: 待计算值。

返回值

double res: value 开平方得到的值

示例

```
res=sqrt(9)                --res=3
```

3.3.5.7 d2r

描述

角度转弧度。

参数

double deg: 角度值。

返回值

double rad: 弧度值

示例

```
rad=d2r(180)               --rad= $\pi$ 
```

3.3.5.8 r2d

描述

弧度转角度。

参数

double rad: 弧度值。

返回值

double deg: 角度值。

示例

```
deg=r2d(math.pi)          --deg=180
```

3.3.5.9 length

描述

计算输入字符串或数组长度。

参数

string/table array: 字符串或数组。

返回值

int len: 字符串或数组的长度。

示例

```
len=length("abcd")          --len=4
```

3.3.5.10 Lua math 库

Lua math 库见表 9。

表 9 Lua math 库

函数名	描述	示例	结果
pi	圆周率	math.pi	3.141592654
abs	取绝对值	math.abs(-2021)	2021
ceil	向上取整	math.ceil(9.1)	10
floor	向下取整	math.floor(9.9)	9
max	取参数最大值	math.max(2,4,6,8)	8
min	取参数最小值	math.min(2,4,6,8)	2
sqrt	开平方	math.sqrt(65536)	256
modf	取整数和小数部分	math.modf(20.12)	20 0.12
randomseed	设随机数种子	math.randomseed(os.time())	
random	取随机数	math.random(5,90)	5~90
rad	角度转弧度	math.rad(180)	3.141592654
deg	弧度转角度	math.deg(math.pi)	180
exp	e 的 x 次方	math.exp(4)	54.59815003
log	计算 x 的自然对数	math.log(54.598150033144)	4
sin	正弦	math.sin(math.rad(30))	0.5
cos	余弦	math.cos(math.rad(60))	0.5
tan	正切	math.tan(math.rad(45))	1
asin	反正弦	math.deg(math.asin(0.5))	30
acos	反余弦	math.deg(math.acos(0.5))	60
atan	反正切	math.deg(math.atan(1))	45
huge	最大值	math.huge	inf
tointeger	把数值强转为整型值	math.tointeger(2.0)	2
type	获取数值的类型	math.type(2)	"integer"
ult	参数以无符号整数形式比较， 第一个参数小于第二个参数时 返回布尔真否则返回假	math.ult(1, 3)	true

3.4 输入输出函数

3.4.1 通讯变量

3.4.1.1 read_int16_register

描述

读 16 位整形寄存器。

参数

int index。

返回值

int Int16Register[index]。

示例

```
a=read_int16_register(1)           --读取 16 位整形寄存器值并赋值给 a
```

3.4.1.2 read_integer_register

描述

读 32 位整形寄存器。

参数

int index。

返回值

int IntegerRegister[index]。

示例

```
a=read_integer_register(1)         --读取 32 位整形寄存器值并赋值给 a
```

3.4.1.3 read_float_register

描述

读浮点型寄存器。

参数

int index。

返回值

float FloatRegister[index]。

示例

```
a=read_float_register(1)
```

```
--读取浮点型寄存器值并赋值给 a
```

3.4.1.4 read_boolean_register

描述

根据输入的索引值，返回 bool 型寄存器的对应值，返回参数类型为 bool 型。

参数

int index, 为 bool 型寄存器索引值，取值范围（0-999）。

返回值

bool BooleanRegister[index]。

示例

```
x_bool=read_boolean_register(1)
```

```
--读取 bool 型寄存器索引值为 1 的内容，执行结果：x_bool 为 true 或者 false
```

3.4.1.5 write_int16_register

描述

写 16 位整形寄存器。

参数

int index;

int16 value。

示例

```
write_int16_register(0,123)
```

```
--写 16 位整形寄存器（偏移地址，值）
```

3.4.1.6 write_integer_register

描述

写 32 位整形寄存器。

参数

int index;

int value。

示例

```
write_integer_register(1,123)
```

```
--写 32 位整形寄存器（偏移地址，值）
```

3.4.1.7 write_float_register

描述

写浮点型寄存器。

参数

int index;

float value。

示例

```
write_float_register(2,123)
```

--写浮点型寄存器（偏移地址，值）

3.4.1.8 write_boolean_register

描述

写 bool 型寄存器。

参数

int index;

bool value。

示例

```
write_boolean_register(3,true)
```

3.4.2 IO 变量

3.4.2.1 set_standard_digital_out

描述

设置标准数字输出。

参数

int index;

bool value（true false）。

示例

```
set_standard_digital_out(0,true)
```

--设置标准数字输出（偏移地址，值）

3.4.2.2 get_standard_digital_out

描述

获取标准数字输出值。

参数

int index。

返回值

bool value。

示例

```
a=get_standard_digital_out(1)           --获取标准数字输出值（偏移地址）并赋值给 a
```

3.4.2.3 get_standard_digital_in

描述

获取标准数字输入值。

参数

int index。

返回值

bool value。

示例

```
a=get_standard_digital_in(0)           --获取标准数字输入值（偏移地址）并赋值给 a
```

3.4.2.4 set_configurable_digital_out

描述

设置可配置数字输出值。

参数

int index;

bool value（true false）。

示例

```
set_configurable_digital_out(1,true)    --设置可配置数字输出值并赋值给 a（偏移地址，值）
```

3.4.2.5 get_configurable_digital_out

描述

获取可配置数字输出值。

参数

int index。

返回值

bool value。

示例

```
a=get_configurable_digital_out(2)      --获取可配置数字输出值（偏移地址）并赋值给 a
```

3.4.2.6 get_configurable_digital_in

描述

获取可配置数字输入值。

参数

int index。

返回值

bool value。

示例

```
a=get_configurable_digital_in(0)      --获取可配置数字输入值（偏移地址）并赋值给 a
```

3.4.2.7 set_standard_analog_out

描述

设置主板模拟量输出值。

参数

int index;

float value。

示例

配置模拟量输出单位为电流。

```
set_standard_analog_out(0,12)      --设置主板模拟量输出值（偏移地址，值）
```

3.4.2.8 get_standard_analog_in

描述

获取主板模拟量输入值。

参数

int index。

返回值

float value。

示例

```
a=get_standard_analog_in(0)          --获取主板模拟量输入值（偏移地址）
```

3.4.2.9 set_tool_digital_out

描述

设置工具法兰数字量输出值。

参数

int index;

bool value。

示例

```
set_tool_digital_out(1,true)         --设置工具法兰数字量输出值（偏移地址，值）
```

3.4.2.10 get_tool_digital_in

描述

获取工具法兰数字量输入值。

参数

int index。

返回值

bool value。

示例

```
a=get_tool_digital_in(0)             --获取工具法兰数字量输入值（偏移地址）
```

3.4.2.11 get_tool_analog_in

描述

获取工具法兰模拟量输入值。

参数

int index。

返回值

float value。

示例

```
a=get_tool_analog_in(0)
```

```
--获取工具法兰模拟量输入值（偏移地址）
```

3.4.2.12 get_tool_digital_out

描述

获取工具法兰数字量输出值。

参数

int index。

返回值

bool value。

示例

```
a=get_tool_digital_out(0)
```

```
--获取工具法兰数字量输出值（偏移地址）
```

3.4.2.13 ctrl_tool_24v_out

描述

控制工具端 24V 电源输出。

参数

bool powerCtrl: true 表示打开电源输出，false 表示关闭电源输出。

返回值

无。

示例

```
ctrl_tool_24v_out(true)
```

3.4.3 弹窗函数

3.4.3.1 str_cat

描述

字符串拼接。

参数

string prefix_str 拼接前缀字符串，可以为空字符串。

第二参数

可以是字符串、数字、布尔变量或常量。

返回值

string value 拼接字符串。

示例

```
retstring = str_cat("this is int = ", 1)           --数字
retstring = str_cat("this is bool = ", true)       --布尔变量
var1 = "lua"
retstring = str_cat("this is str var = ", var1)    --字符串
```

3.4.3.2 popup_message

描述

输出弹窗。

参数

string msg 弹窗消息文本；

string title 弹窗标题文本；

bool warn 警告弹窗类型，true 表示弹窗类型选择警告弹窗；

bool err 错误弹窗类型，true 表示弹窗类型选择错误弹窗。

示例

```
popup_message( str_cat("", "this is lua pop"), "popup example", false, false)
--弹窗消息 this is lua pop, 弹窗标题 popup example
```

3.4.3.3 popin_boolean

描述

布尔类型输入弹窗。

参数

string title 弹窗标题文本。

返回值

bool value 输入布尔值。

示例

```
ret_bool = popin_boolean("popin example") --布尔类型输入弹窗，弹窗名为 popin example
```

3.4.3.4 popin_number

描述

数字类型输入弹窗。

参数

string title 弹窗标题文本。

返回值

number value 输入数字。

示例

```
ret_number = popin_number("popin example")    --数字类型输入弹窗，弹窗名为 popin example
```

3.4.3.5 popin_string

描述

字符串类型输入弹窗。

参数

string title 弹窗标题文本。

返回值

string value 输入字符。

示例

```
ret_string = popin_string("popin example")    --字符串类型输入弹窗，弹窗名为 popin example
```

3.4.4 寄存器变量

3.4.4.1 get_sys_freq_float

描述

读 SFD 寄存器浮点值。

参数

int index: 寄存器地址。

返回值

float res: 寄存器值。

示例

```
res = get_sys_freq_float(1000)
```

3.4.4.2 get_sys_freq_int

描述

读 SFD 寄存器双字值。

参数

int index: 寄存器地址。

返回值

int res: 寄存器值。

示例

```
res = get_sys_freg_int(1000)
```

3.4.4.3 get_sys_wreg_float

描述

读 SD 寄存器浮点值。

参数

int index: 寄存器地址。

返回值

float res: 寄存器值。

示例

```
res = get_sys_wreg_float(1000)
```

3.4.4.4 get_sys_wreg_short

描述

读 SD 寄存器单字值。

参数

int index: 寄存器地址。

返回值

short res: 寄存器值。

示例

```
res = get_sys_wreg_short(1000)
```

3.4.4.5 get_sys_wreg_int

描述

读 SD 寄存器双字值。

参数

int index: 寄存器地址。

返回值

int res: 寄存器值。

示例

```
res = get_sys_wreg_short(1000)
```

3.4.4.6 get_wreg_float**描述**

读 D 寄存器浮点值。

参数

int index: 寄存器地址。

返回值

float res: 寄存器值。

示例

```
res = get_wreg_float(1000)
```

3.4.4.7 get_wreg_short**描述**

读 D 寄存器单字值。

参数

int index: 寄存器地址。

返回值

short res: 寄存器值。

示例

```
res = get_wreg_short(1000)
```

3.4.4.8 get_wreg_int**描述**

读 D 寄存器双字值。

参数

int index: 寄存器地址。

返回值

int res: 寄存器值。

示例

```
res = get_wreg_int(1000)
```

3.4.4.9 get_sys_hwreg_float

描述

读 HSD 寄存器浮点值。

参数

int index: 寄存器地址。

返回值

float res: 寄存器值。

示例

```
res = get_sys_hwreg_float(1000)
```

3.4.4.10 get_sys_hwreg_short

描述

读 HSD 寄存器单字值。

参数

int index: 寄存器地址。

返回值

short res: 寄存器值。

示例

```
res = get_sys_hwreg_short(1000)
```

3.4.4.11 get_sys_hwreg_int

描述

读 HSD 寄存器双字值。

参数

int index: 寄存器地址。

返回值

int res: 寄存器值。

示例

```
res = get_sys_hwreg_int(1000)
```

3.4.4.12 get_sys_breg**描述**

读 SM 寄存器位值。

参数

int index: 寄存器地址。

返回值

bool res: 寄存器值。

示例

```
res = get_sys_breg(3150)
```

3.4.4.13 set_sys_freg_float**描述**

写 SFD 寄存器浮点值。

参数

int index: 寄存器地址;

float value: 寄存器值。

返回值

无。

示例

```
set_sys_freg_float(10000,0.0)
```

3.4.4.14 set_sys_freg_int**描述**

写 SFD 寄存器双字值。

参数

int index: 寄存器地址;

int value: 寄存器值。

返回值

无。

示例

```
set_sys_freg_int(10000,0)
```

3.4.4.15 set_sys_freg_short

描述

写 SFD 寄存器单字值。

参数

int index: 寄存器地址;

int value: 寄存器值。

返回值

无。

示例

```
set_sys_freg_short(10000,0)
```

3.4.4.16 set_sys_wreg_float

描述

写 SD 寄存器浮点值。

参数

int index: 寄存器地址;

float value: 寄存器值。

返回值

无。

示例

```
set_sys_wreg_float(10000,0.0)
```

3.4.4.17 set_sys_wreg_short

描述

写 SD 寄存器单字值。

参数

int index: 寄存器地址;

short value: 寄存器值。

返回值

无。

示例

```
set_sys_wreg_short(10000,0)
```

3.4.4.18 set_sys_wreg_int**描述**

写 SD 寄存器双字值。

参数

int index: 寄存器地址;

int value: 寄存器值。

返回值

无。

示例

```
set_sys_wreg_int(10000,1)
```

3.4.4.19 set_wreg_float**描述**

写 D 寄存器浮点值。

参数

int index: 寄存器地址;

float value: 寄存器值。

返回值

无。

示例

```
set_wreg_float(10000,0.0)
```

3.4.4.20 set_wreg_short

描述

写 D 寄存器单字值。

参数

int index: 寄存器地址;

short value: 寄存器值。

返回值

无。

示例

```
set_wreg_short(10000,0)
```

3.4.4.21 set_wreg_int

描述

写 D 寄存器双字值。

参数

int index: 寄存器地址;

int value: 寄存器值。

返回值

无。

示例

```
set_wreg_int(10000,1)
```

3.4.4.22 set_sys_hwreg_float

描述

写 HSD 寄存器浮点值。

参数

int index: 寄存器地址;

float value: 寄存器值。

返回值

无。

示例

```
set_sys_hwreg_float(10000,0.0)
```

3.4.4.23 set_sys_hwreg_short

描述

写 HSD 寄存器单字值。

参数

int index: 寄存器地址;

short value: 寄存器值。

返回值

无。

示例

```
set_sys_hwreg_short(10000,0)
```

3.4.4.24 set_sys_hwreg_int

描述

写 HSD 寄存器双字值。

参数

int index: 寄存器地址;

int value: 寄存器值。

返回值

无。

示例

```
set_sys_hwreg_int(10000,1)
```

3.4.4.25 set_sys_breg

描述

写 SM 寄存器位值。

参数

int index: 寄存器地址;

bool value: 寄存器值。

返回值

无。

示例

```
set_sys_breg(3152,1)
```

3.5 通讯函数

3.5.1 Modbus 通讯

参数说明：

- slaveNo: 从站站号。
- addr: modbus 寄存器地址。
- num: 读取的寄存器个数。
- comNo: 端口号--RS232==1,RS485-1==2 RS485-2 ==3, 工具端==6, 以太网口==502。
- regtype: 通讯变量类型: 0 为 bool 变量, 1 为 int16, 2 为 int32, 3 位 float, 线圈操作默认为 bool

型变量, 此参数不需要。

- regoffset: 通讯变量起始偏移量。
- socketID: 结合 connect 创建的 socketid 号, 非以太网口, 此参数不需要。

3.5.1.1 modbus_read_bits

描述

modbus 读位寄存器。

参数

int slaveNo;

int addr;

int num;

int comNo;

int destBoolIndex;

int socketID。

示例

```
modbus_read_bits(1,100,6,2,0)
```

```
--modbus 读位寄存器，从站 1，寄存器地址 100，读取 6 个寄存器，端口 RS485-1，此处通讯变量  
类型为 bool 变量（可缺省）  
  
--如果端口号是 502，需要设置 socketID 和 socket 连接  
  
modbus_read_bits(1,100,6,502,0,10)
```

3.5.1.2 modbus_read_input_bits

描述

modbus 读输入位寄存器（只读不写）。

参数

int slaveNo;

int addr;

int num;

int comNo;

int destBoolIndex;

int socketID。

示例

```
modbus_read_input_bits(1,100,6,2,0,10)  
  
--modbus 读输入位寄存器，从站 1，寄存器地址 100，读取 6 个寄存器，端口 RS485-1，此处通讯  
变量类型为 bool 变量（可缺省）  
  
--如果端口号是 502，需要设置 socketID 和 socket 连接  
  
modbus_read_input_bits(1,100,6,502,0,10)
```

3.5.1.3 modbus_write_bit

描述

modbus 写位寄存器。

参数

int slaveNo;

int addr;

int comNo;

int destBoolIndex;

int socketID。

示例

```
modbus_write_bit(1,100,2,0)
```

--modbus 写位寄存器，从站 1，寄存器地址 100，端口 RS485-1,此处通讯变量类型为 bool 变量（可缺省）

--如果端口号是 502，需要设置 socketID 和 socket 连接，socketID 为 10

```
modbus_write_bit(1,100,502,0,10)
```

3.5.1.4 modbus_write_bits

描述

modbus 写多个位寄存器。

参数

int slaveNo;

int addr;

int num;

int comNo;

int destBoolIndex;

int socketID。

示例

```
modbus_write_bits(1,100,8,2,0)
```

--modbus 写多个位寄存器，从站 1，寄存器地址 100，写 8 个数据，端口 RS485-1,此处通讯变量类型为 bool 量（可缺省）

--如果端口号是 502，需要设置 socketID 和 socket 连接，socketID 为 10

```
modbus_write_bits(1,100,8,502,0,10)
```

3.5.1.5 modbus_read_registers

描述

modbus 读寄存器。

参数

int slaveNo;

```
int addr;  
  
int num;  
  
int comNo;  
  
int regtype;  
  
int destRegIndex;  
  
int socketID。
```

示例

```
modbus_read_registers(1,1000,1,502,1,10,5)  
  
--将对方 int16(0)开始的 1 个寄存器通过网线读取到本地 int16(10)  
  
--param1: 站号  
  
--param2: 对方开始寄存器地址，如读取另一台控制器 int16(0)，对应《控制器 MODBUS 地址表》，  
地址为 1000  
  
--param3: 通讯变量个数  
  
--param4: 网络通讯-502  
  
--param5: 通讯变量类型：0 为 bool 变量，1 为 int16，2 为 int32，3 为 float  
  
--param6: 本地通讯变量起始偏移量，如通讯变量类型为 1，偏移量 10 即对应 int16(10)  
  
--param7 SOCK ID (和 socket_connect 相同)
```

3.5.1.6 modbus_read_input_registers

描述

modbus 读输入寄存器（只读不写）。

参数

```
int slaveNo;  
  
int addr;  
  
int num;  
  
int comNo;  
  
int regtype;  
  
int destRegIndex;  
  
int socketID。
```

示例

```
modbus_read_input_registers(1,100,8,2,1,10) --modbus 读输入寄存器，从站 1，寄存器地址 100，读 8 个数据，端口 RS485-1，int16 类型，寄存器起始偏移量为 10
```

```
modbus_read_input_registers(1,100,8,502,1,10,5) --modbus 读输入寄存器，从站 1，寄存器地址 100，读 8 个数据，端口 502，int16 类型，寄存器起始偏移量为 10
```

3.5.1.7 modbus_write_register

描述

modbus 写单个寄存器。

参数

int slaveNo;

int addr;

int comNo;

int regtype;

int destRegIndex;

int socketID。

示例

```
modbus_write_register(1,100,2,1,10)
```

--modbus 写单个寄存器，从站 1，寄存器地址 100，端口 RS485-1，int16 类型，寄存器起始偏移量为 10

```
modbus_write_register(1,100,502,1,10,5)
```

--modbus 写单个寄存器，从站 1，寄存器地址 100，端口 502，int16 类型，寄存器起始偏移量为 10

3.5.1.8 modbus_write_registers

描述

写多个寄存器。

参数

int slaveNo;

int addr;

```
int num;  
  
int comNo;  
  
int regtype;  
  
int destRegIndex;  
  
int socketID。
```

示例

```
modbus_write_registers(1,100,8,2,1,10)
```

--modbus 写多个寄存器，从站 1，寄存器地址 100，写 8 个数据，端口 RS485-1，int16 类型，寄存器起始偏移量为 10

```
modbus_write_registers(1,100,8,502,1,10,5)
```

--modbus 写多个寄存器，从站 1，寄存器地址 100，写 8 个数据，端口 502，int16 类型，寄存器起始偏移量为 10

3.5.2 网口通讯

3.5.2.1 socket_connect

描述

作为客户端连接服务器。

参数

```
const char *TCPorUDP;
```

```
const char *ip;
```

```
int SocketID;
```

```
int portNo。
```

示例

```
socket_connect("tcp","192.168.6.201",8,9000)
```

--descript: 请求控制器创建一个客户端

--param1: 协议类型 "TCP" (支持最多 16 个) 或者 "UDP"(支持最多 8 个)

--param2: 服务端 IP 地址，例如 "192.168.6.100"

--param2: SOCKET ID 取值范围 2-23

--param3: SOCKET PORT 端口号 与服务端相同

3.5.2.2 socket_disconnect

描述

作为客户端断开连接。

参数

int SocketID。

示例

```
socket_disconnect(8)           --断开 SocketID 为 8 的连接
```

3.5.2.3 socket_open

描述

作为服务器端打开端口。

参数

const char *TCPorUDP;

int SocketID;

int portNo。

示例

```
socket_open("tcp",8,9000)      --descript: 请求控制器打开控制器端口

--param1: 协议类型 "TCP" (支持最多 16 个) 或者 "UDP"(支持最多 8 个)

--param2: SOCKET ID 取值范围 2-23

--param3: SOCKET PORT 端口号
```

3.5.2.4 socket_close

描述

作为服务器端关闭端口。

参数

int SocketID。

示例

```
socket_close(8)               --关闭 SocketID 为 8 的端口
```

3.5.2.5 socket_receive

描述

接收数据并写入指定通讯类型变量的起始偏移地址处；接收完成后可调用 `get_last_receive_status()` 函数获取接收的结果。假如发送过来的十六进制数据为“12 34 56 78 AB CD”，则接收到的 16 位整数寄存器十六进制为“12 34 56 78 AB CD”。

参数

`int socketID;`

`int num` 通讯变量个数；

`CommuVarType regType` 通讯变量类型：0 为 bool 变量，1 为 int16，2 为 int32，3 为 float；

`int destRegIndex` 通讯变量起始偏移量；

`int timeout` 接收超时时间，单位毫秒；若 ≤ 0 ，无限等待直到接收到数据。

示例

```
socket_receive(8,1000,1,0,2000)           --接收数据
--param1: SOCKET ID
--param2: 通讯变量个数
--param3: 通讯变量类型：0 为 bool 变量，1 为 int16，2 为 int32，3 为 float
--param4: 通讯变量起始偏移量
--param5: 超时时间 ms
```

3.5.2.6 socket_send

描述

网口数据发送。

参数

`int socketID;`

`int num;`

`CommuVarType regType;`

`int destRegIndex。`

示例

```
socket_send(8,1000,1,0)                   --发送指定通讯类型变量地址处的数据
--param1: SOCKET ID
--param2: 通讯变量个数
```

```
--param3: 通讯变量类型: 0 为 bool 变量, 1 为 int16, 2 为 int32, 3 为 float
--param4: 通讯变量起始偏移量
```

3.5.2.7 socket_receive_string

描述

收字符串并写入指定类型的通讯变量地址处; 接收完成后可调用 `get_last_receive_status()` 函数获取接收状态。假如发送过来的十六进制数据为 “12 34 56 78 AB CD”，则接收到的 16 位整数寄存器十六进制数据为 “34 12 78 56 CD AB”。

参数

int socketID;

int num 接收字符最大个数;

CommuVarType regType 通讯变量类型: 0 为 bool 变量, 1 为 int16, 2 为 int32, 3 为 float;

int destRegIndex 通讯变量起始偏移量;

int timeout 接收超时时间, 单位毫秒; 若 ≤ 0 , 无限等待直到接收到数据。

示例

```
socket_receive_string(6,8,1,0,2000)

--接收 8 个字符的数据, 接收到后从 int16 通讯变量寄存器的偏移地址 0 开始依次存入。

--param1: SOCKET ID
--param2: 接收字符个数
--param3: 通讯变量类型: 0 为 bool 变量, 1 为 int16, 2 为 int32, 3 为 float
--param4: 通讯变量起始偏移量
--param5: 接收超时时间 ms

recv_status = get_last_receive_status()
if(recv_status == 1)
then
    str = commuvar_to_string(8,1,0)
    print(str)
else
    print("recv fail")
```

```
end
```

3.5.2.8 socket_send_string

描述

可直接发送字符串至指定端口也可发送指定字符串至指定端口并存放在指定通讯类型变量地址处。

参数

int socketID;

CommuVarType regType 通讯变量类型：0 为 bool 变量，1 为 int16，2 为 int32，3 为 float;

int destRegIndex 通讯变量起始偏移量;

const char *str 发送字符串。

示例

```
socket_send_string(6,1,1,"(6,1,2,3,4,5,7)")
```

--socketID 为 6，通讯类型为 int16，起始偏移量为 1，发送字符串(6,1,2,3,4,5,7)

--param1: SOCKET ID

--param2: 通讯变量类型：0 为 bool 变量，1 为 int16，2 为 int32，3 为 float

--param3: 通讯变量起始偏移量

--param4: 字符串

```
socket_send_string(6,"(6,1,2,3,4,5,7)") --socketID 为 6，发送字符串(6,1,2,3,4,5,7)
```

3.5.2.9 socket_read_string

描述

读取指定端口的字符串。

参数

int socketID;

int timeout 接收超时时间，单位毫秒；若<=0，无限等待直到接收到数据。

示例

```
str = socket_read_string(6,2000)
```

--从 socketID 为 6 的端口读取字符串并赋值给变量 str，接收超时时间为 2000ms

3.5.3 串口通讯

3.5.3.1 serial_receive

描述

接收数据并写入指定通讯类型变量的起始偏移地址处；接收完成后可调用 `get_last_receive_status()` 函数获取接收的结果。

参数

int comNo 端口号：RS232 == 1, RS485-1 == 2, RS485-2 == 3, 工具端 == 6;

int num 通讯变量个数;

CommuVarType regType 通讯变量类型：0 为 bool 变量, 1 为 int16, 2 为 int32, 3 为 float;

int destRegIndex 通讯变量起始偏移量;

int timeout 接收超时时间, 单位毫秒; 若 ≤ 0 , 无限等待直到接收到数据。

示例

```
serial_receive(2,1,1,0,2000)
```

--从 RS485-1 口接收 1 个 int16 类型的数据, 接收到后存入 int16 通讯变量寄存器的偏移地址 0 处。

备注：接收超时除 timeout 参数值外, 还受串口配置的超时时间参数影响; 实际超时时间以两个超时参数中小的为最终超时时间。

3.5.3.2 serial_send

描述

串口数据发送。

参数

int comNo 端口号：RS232 == 1, RS485-1 == 2, RS485-2 == 3, 工具端 == 6;

int num 通讯变量个数;

CommuVarType regType 通讯变量类型：0 为 bool 变量, 1 为 int16, 2 为 int32, 3 为 float;

int destRegIndex 通讯变量起始偏移量。

示例

```
serial_send(2,1,1,3)
```

--通过 RS485-1 口发送一个 int16 类型的数据, 该数据位于通讯变量寄存器的偏移地址 3

备注：函数为阻塞类型, 即需要等到发送完成后, 后续脚本才能继续执行

3.5.4 QEP 正交脉冲计数

3.5.4.1 encoder_enable

描述

使能 encoder。

参数

int ChannelNo: 编码器通道号: 0-QEP0, 1-QEP1;

int trigsel: 选通信号触发方式: 0-上升沿触发, 1-下降沿触发。

返回值

无。

示例

```
encoder_enable(0,1)
```

--使能通道 0 计数, 选通信号配置为下降沿触发; 一般应放在执行前序列中

3.5.4.2 encoder_disable

描述

禁止 encoder。

参数

int ChannelNo: 编码器通道号: 0-QEP0, 1-QEP1。

返回值

无。

示例

```
encoder_disable(0)
```

3.5.4.3 encoder_get_tick_count

描述

获取编码器脉冲数。

参数

int ChannelNo: 编码器通道号: 0-QEP0, 1-QEP1。

返回值

long long tickCount: 编码器总脉冲数(8 字节)。

示例

```
number=encoder_get_tick_count(1)           --获取通道为 1 的编码器脉冲数
```

3.5.4.4 encoder_get_strobe_tick_count

描述

获取选通信号触发脉冲数。

参数

int ChannelNo: 编码器通道号: 0-QEP0, 1-QEP1。

返回值

long long tickCount: 编码器总脉冲数(8 字节)。

备注: 选通信号功能需要将 HSD901 置 1 后重新上电生效;

编码器通道 0 选通信号 - DI4 上升沿有效;

编码器通道 1 选通信号 - DI5 上升沿有效。

示例

```
number=encoder_get_strobe_tick_count(1)     --获取通道为 1 的选通信号触发脉冲数
```

3.5.4.5 encoder_set_tick_count

描述

设置编码器脉冲数。

参数

int ChannelNo: 编码器通道号, 0-QEP0, 1-QEP1。

long long Value: 设置编码器脉冲数为 Value。

示例

```
encoder_set_tick_count(0,5555555555)
--将通道 0 得编码器脉冲数设置为 5555555555
```

3.5.5 通用通讯函数

3.5.5.1 get_last_receive_status

描述

获取最近接收通讯函数的通讯状态。

参数

无。

返回值

int status: 通讯结果, 0-未接收到数据, 1-成功接收数据。

示例

```
recv_status = get_last_receive_status() --获取最近接收通讯函数得通讯状态
```

3.5.5.2 get_socket_connect_status

描述

用来获取 Socket Connect 的状态。

参数

int socketID。

返回值

int status: connect status, 0-连接成功, 1-连接失败, 2-连接中, 3-非法连接。

示例

```
connect_status = get_socket_connect_status(1) --用来获取从站 1 的 Socket Connect 状态
```

3.5.6 ethercat 通讯

3.5.6.1 read_ect_pdo

描述

读取 txpdo 实时数据, 通过对应配置的偏移地址 (offset) 读取对应 pdo 参数值。

参数

int offset。

返回值

number value, 根据实际配置的数据类型返回。

示例

```
a=read_ect_pdo(201) --读取偏移地址为 201 的值, 赋值给 a
```

3.5.6.2 write_ect_pdo

描述

设置 Rxpdo 参数, 通过对应配置的偏移地址 (offset) 写入对应 pdo 参数。

注: 关节对应的 pdo 参数不可写入。

参数

int offset;

number value 根据实际配置的数据类型转换写入。

示例

```
write_ect_pdo(108,10)
```

--偏移地址为 108 写入参数为 10

3.5.6.3 read_ect_sdo

描述

读取 SDO 参数。

参数

int slaveIndex: 从站号;

int SDOIndex: 十六进制 SDO 索引;

int SDOSubIndex: 十六进制 SDO 子索引。

返回值

number data: 读到的 SDO 值。

示例

```
a = read_ect_sdo(1,0x2023,0x00)
```

--读取从站 1 索引为 0x2023、子索引为 0 的 SDO 值，赋值给 a

3.5.6.4 write_ect_sdo

描述

写入 SDO 参数。

参数

int slaveIndex: 从站号;

int SDOIndex: 十六进制 SDO 索引;

int SDOSubIndex: 十六进制 SDO 子索引;

number value: 要写入的值。

返回值

无。

示例

```
write_ect_sdo(1,0x2023,0x00,1)
```

--将从站 1 索引为 0x2023、子索引为 0x00 的 SDO 写入值 1

3.5.7 辅助功能

3.5.7.1 read_masterboard_mb_reg

描述

读取控制器 modbus 寄存器 10-999 之间的数据。

参数

int addr: 寄存器地址（10-999）。

返回值

number value: 返回目标地址的寄存器数据（16 位）。

示例

```
a = read_masterboard_mb_reg(220)    --读取控制器 modbus 地址为 220 的数据，赋值给 a
```